SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA37 – Software Testing

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Black-Box and White-Box Test Case Design Techniques for a Cab Booking and Fare Calculation System (SwiftRide)

CO2: Design effective test cases using black-box and white-box testing techniques and control-flow-based methods.

Student Name	SAJAN S	**Register Number**	1911260014
Department / Section	CSE	**Academic Year / Semester**	2026-27
Faculty Name	Dr.Gnanajeyaraman	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Test Case Design Strategies: Specification, Structure, Fault-based	4		
2	Random Testing: Uniform, Operational-profile, Adaptive	4		
3	Equivalence Class Partitioning Variants	5		
4	Boundary Value Analysis Methods	5		
5	Decision Table, State-Transition and Pairwise Testing	5		
6	White-Box Testing and Test Adequacy Criteria	4		
7	Coverage: Decision, Condition/Decision, MC/DC	5		
8	Control Flow Graphs: Statement, Block, DD-path	4		
9	Path Testing and the Integrated Test Design	6		
10	Re-apply and Adapt the Test Design	8		
	TOTAL	**50**		

Submitted by ARAVINDH R (192312024)

Contents

0. Scope, Environment and Assumptions
Task 1 – Test Case Design Strategies: Specification-Based, Structure-Based and Fault-Based Design
Task 2 – Random Testing: Uniform, Operational-Profile and Adaptive Random Generation
Task 3 – Equivalence Class Partitioning: Valid-Class, Robust and Classification-Tree Variants
Task 4 – Boundary Value Analysis: Two-Value, Robust and Worst-Case Boundary Testing
Task 5 – Decision Table, State-Transition and Pairwise Testing
Task 6 – White-Box Testing and Test Adequacy: Statement, Branch and Mutation-Score Criteria
Task 7 – Coverage Criteria: Decision, Condition/Decision and MC/DC
Task 8 – Control Flow Graphs: Statement-Level, Basic-Block and DD-Path Graphs of find_driver
Task 9 – Path Testing and the Integrated Test Design
Task 10 – Re-apply and Adapt the Test Design to the Changed Requirement
Appendix A – Evidence of Execution (Tool Output)
Appendix B – Mapping of the Report to the Assignment Requirements
Appendix C – Limitations and Honest Notes
Appendix D – Files Delivered with this Report

Reading guide. Every table and chart in this report is the output of code that was run; the figures (control flow graphs, state diagram, workflow) are drawn with Graphviz and the charts with matplotlib. Sections “Prediction recorded for Task 9” are written before the Task 9 results so that Task 9 can test them.

0. Scope, Environment and Assumptions

This report answers all ten tasks of Software Testing Assignment 2 (SwiftRide cab booking and fare engine). Listing 1 (calculate_fare) and Listing 2 (find_driver) were implemented exactly as printed (file listing1.py has the same 20 lines and listing2.py the same 12 lines, so every line number quoted below is the line number of the assignment). Every faults-found figure, coverage percentage, mutation score, V(G) value, pairwise size and run time in this report was produced by running the code described in Appendix A; nothing is copied from another source.

0.1 Environment

Item	Value
Language / tools	Python 3.12.3, pytest 9.2.0, coverage.py 7.17.1 (branch mode), Hypothesis 6.170.14, radon 6.0.1, allpairspy (pairwise), Graphviz (diagrams)
Code under test	listing1.py = Listing 1; listing2.py = Listing 2; ride.py = ride life-cycle class (Task 5); fare2_src.py = extended fare function (Task 10)
Fault injection	A mutant is the Listing 1 source text with one edit (or a small group of edits) made by common.mutate(); it is compiled and run beside the original
Detection rule	A seeded fault is detected when a test makes the mutant return a different value from the original (differential check). For the specification tests the expected value is also given by an independent Decimal-based oracle (common.spec_oracle) written only from the specification
Hardware / timing	Single process on a cloud container; run times are in-process times of the function. Where a project-level run time is needed (Task 10) an explicit assumption of 0.5 s per automated test through the service layer is used

0.2 Additional assumptions (stated and justified)

- **Waiting time and booking hour are integers** (minutes and clock hours); distance is on a 0.1 km grid (given). The wallet balance is a whole number of rupees.
- **Night rule in the oracle:** hours 23 and 0–4 (up to 4:59 am) get the 1.25 factor. This is identical to Listing 1 (hour >= 23 or hour < 5) for integer hours.
- **Expected value for invalid wait, hour, wallet or mode:** the specification only declares distance outside 0.1–60 km INVALID. For the other parameters a robust system should also reject out-of-range values, so the strict oracle expects INVALID; Task 3 shows Listing 1 does not do this and reports it as a robustness defect, not as a seeded fault.
- **Design effort** (minutes) in the comparison tables is a planning estimate made by the author from the work involved (reading the rules, writing the tests, checking expected values); it was not measured with a stopwatch.
- **LLM critique parts.** Where a task asks for an LLM to be used, the LLM output shown is a single one-pass answer from Claude (no prompt tuning, no second attempt) to the prompt quoted in that task. It was then verified by running or by tracing. A different model or a second run may give different output, so these sections describe one run, not the behaviour of all LLMs.

0.3 Important finding that changes the seeded faults

Several of the example faults suggested in the assignment are **equivalent mutants** and can never be detected, whatever the test. Because the fare slabs are continuous (at 2 km both formulas give ₹50 and at 10 km both give ₹146) and because the first 5 minutes of waiting cost 0, the three edits km <= 2 → km < 2, km <= 10 → km < 10 and wait > 5 → wait >= 5 produce exactly the same output as the original. This was checked by running original and mutant over 4,303,728 input combinations (km from −0.5 to 61.6 in 0.1 steps × wait −1 to 31 × hour −1 to 24 × promo × four wallet/mode combinations): zero differences for all three. Therefore the report keeps the three literal edits only as an equivalent-mutant example and uses **shifted-boundary faults** instead (km <= 2 → km <= 2.1, km <= 10 → km <= 10.1, wait > 5 → wait > 6), which are detectable.

Literal edit from the assignment	Differences found in 4,303,728 inputs	Verdict	Detectable replacement used
km <= 2 → km < 2 (line 4)	0	equivalent	km <= 2 → km <= 2.1

Literal edit from the assignment	Differences found in 4,303,728 inputs	Verdict	Detectable replacement used
km <= 10 → km < 10 (line 6)	0	equivalent	km <= 10 → km <= 10.1
wait > 5 → wait >= 5 (line 10)	0	equivalent	wait > 5 → wait > 6

Task 1 – Test Case Design Strategies: Specification-Based, Structure-Based and Fault-Based Design

1.1 Seeded faults

ID	Line	Original	Seeded fault	Type
F1	4	km <= 2	km <= 2.1	shifted boundary
F2	12	hour >= 23	hour > 23	boundary operator
F3	14	fare >= 200	fare >= 150	wrong constant
F4	10	wait > 5	wait > 6	shifted boundary
F5	16–17	800 (cap)	900 (cap)	wrong constant
F6	18	wallet < fare	wallet <= fare	wrong operator

1.2 The 12 test cases designed by each strategy and their execution

Notation: km | w = waiting minutes | h = hour | promo | wallet | mode. “Faults killed” lists the seeded faults that this single test detects.

Strategy A – specification-based (rules only)

#	Input	Expected (oracle)	Faults killed
1	1 km | w0 | h12 | – | 0 | CASH	50	–
2	2 km | w0 | h12 | – | 0 | CASH	50	–
3	2.1 km | w0 | h12 | – | 0 | CASH	51.2	F1
4	10 km | w0 | h12 | – | 0 | CASH	146	–
5	10.1 km | w0 | h12 | – | 0 | CASH	147	–
6	5 km | w5 | h12 | – | 0 | CASH	86	–
7	5 km | w6 | h12 | – | 0 | CASH	88	F4
8	5 km | w0 | h23 | – | 0 | CASH	107.5	F2
9	20 km | w0 | h12 | RIDE50 | 0 | CASH	196	–
10	15 km | w0 | h12 | RIDE50 | 0 | CASH	196	F3
11	60 km | w30 | h23 | – | 0 | CASH	800	F2, F5
12	5 km | w0 | h12 | – | 86 | WALLET	86	F6

Strategy B – structure-based (statements and branches of Listing 1)

#	Input	Expected (oracle)	Faults killed
1	-1 km | w0 | h12 | – | 0 | CASH	INVALID	–
2	61 km | w0 | h12 | – | 0 | CASH	INVALID	–
3	1 km | w0 | h12 | – | 0 | CASH	50	–
4	5 km | w0 | h12 | – | 0 | CASH	86	–
5	20 km | w0 | h12 | – | 0 | CASH	246	–
6	5 km | w10 | h12 | – | 0 | CASH	96	–
7	5 km | w0 | h2 | – | 0 | CASH	107.5	–
8	20 km | w0 | h12 | RIDE50 | 0 | CASH	196	–
9	60 km | w30 | h2 | RIDE50 | 0 | CASH	800	F5
10	5 km | w0 | h12 | – | 10 | WALLET	WALLET_LOW	–
11	5 km | w0 | h12 | – | 500 | WALLET	86	–
12	30 km | w0 | h12 | RIDE50 | 500 | CASH	296	–

Strategy C – fault-based (one test per likely fault in the catalogue)

#	Input	Expected (oracle)	Faults killed
1	0 km | w0 | h12 | – | 0 | CASH	INVALID	–
2	60 km | w0 | h12 | – | 0 | CASH	646	–
3	2.1 km | w0 | h12 | – | 0 | CASH	51.2	F1
4	10.1 km | w0 | h12 | – | 0 | CASH	147	–
5	5 km | w6 | h12 | – | 0 | CASH	88	F4
6	5 km | w5 | h12 | – | 0 | CASH	86	–
7	5 km | w0 | h23 | – | 0 | CASH	107.5	F2
8	5 km | w0 | h5 | – | 0 | CASH	86	–
9	14 km | w12 | h12 | RIDE50 | 0 | CASH	150	–
10	15 km | w0 | h12 | RIDE50 | 0 | CASH	196	F3
11	60 km | w30 | h23 | – | 0 | CASH	800	F2, F5
12	5 km | w0 | h12 | – | 86 | WALLET	86	F6

Strategy A took its values from the rules and their limits (2, 10, 5/6, 23, 200, 800, balance equal to fare). Strategy B chose any convenient value that drives a branch the other way (for example km = 5, 20, 61) because for structure only the branch outcome matters. Strategy C started from the fault list (wrong operator, wrong constant, missing condition) and asked, for each, which input would make the faulty program differ from the correct one; this is the mutation idea used as a design tool.

1.3 Fault-detection table

Strategy	Tests	Faults found	Faults per test	Design effort (min, estimate)	Faults missed
A Specification-based	12	6 / 6	0.50	25	none
B Structure-based	12	1 / 6	0.08	20	F1, F2, F3, F4, F6
C Fault-based	12	6 / 6	0.50	35	none
LLM one-pass suite	17	2 / 6	0.12	2 (prompting) + review	F1, F3, F4, F6

Fault	A	B	C	LLM
F1 km<=2 -> km<=2.1 (slab shift)	found	MISSED	found	MISSED
F2 hour>=23 -> hour>23	found	MISSED	found	found
F3 promo threshold 200 -> 150	found	MISSED	found	MISSED
F4 wait>5 -> wait>6	found	MISSED	found	MISSED
F5 cap 800 -> 900	found	found	found	found
F6 wallet<fare -> wallet<=fare	found	MISSED	found	MISSED

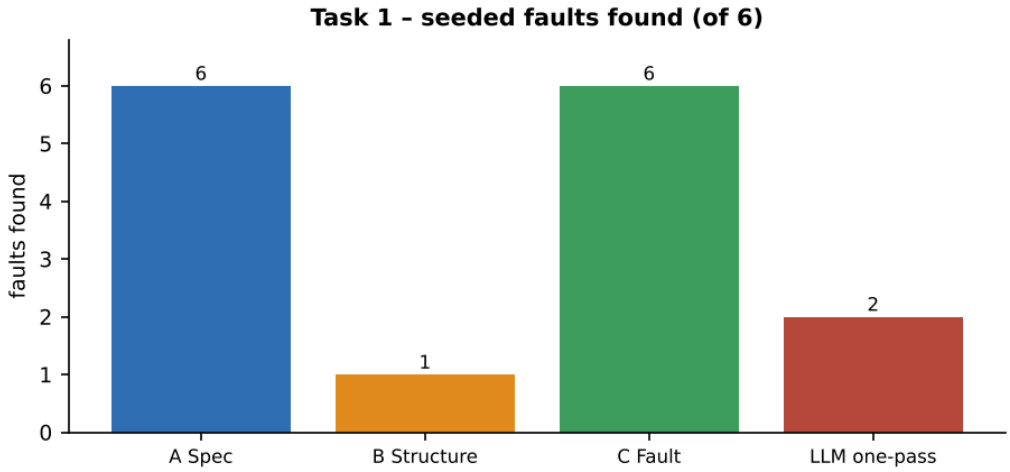


Figure 1.1 – Seeded faults found by each strategy (12 tests each; LLM suite 17 tests)

1.4 What each strategy tends to miss, and why

- **Structure-based (B) found only F5.** The branch outcome of line 14 is reached by any fare ≥ 200 (246 or 696), so the test never needs fare 199 or 150–199 to expose the threshold 150 (F3); line 10 is entered by wait = 10 whatever the threshold (6 or 5) is, so F4 survives; line 12 is entered by hour 2, so F2 (hour 23) survives; line 18 is exercised with wallet 10 and 500 but never with wallet equal to fare, so F6 survives. A structural criterion asks “was this branch taken?”, not “was it taken at the correct limit?”. 100% branch coverage can be reached with all of these faults still present.
- **Specification-based (A) and fault-based (C) found 6 of 6** because both put the values on the limits named by the rules (2.1, 10.1, 6 minutes, hour 23, fare 196 and 246, balance 86 for a fare of 86).
- **Missing functionality** is invisible to B by construction. The specification says “each passenger may use RIDE50 once”, that waiting time is 0–30 and that the hour is 0–23, but Listing 1 has no code for the once-only rule and no range check for wait, hour or mode (Task 3 proves the last point by execution). A test set derived from the code has nothing to execute for code that is not there. Only a specification-based test (“use RIDE50 a second time → rejected”) can fail.
- **Extra or wrong code** (code that does something the specification never asked for) is where B and C are better than A: A has no reason to test a branch the specification does not mention, while coverage measurement shows an unexpected branch and mutation shows an unexpected operator.
- In this experiment the missing-functionality argument is not an abstract claim: of the 8 invalid classes tried in Task 3, Listing 1 correctly rejects only the 2 distance classes and silently accepts 6 others.

1.5 Dependence on different inputs, and incomplete requirements

Strategy	Input it needs	If the requirement document is incomplete	If the code changes
A Specification	Fare, waiting, night, promo, cap and payment rules	Directly damaged: a missing rule (for example, what happens to a wallet of exactly the fare) leaves a gap; testers must guess or ask	Unchanged if rules unchanged; tests stay valid after refactoring
B Structure	Listing 1 source	Hardly affected for the code that exists, but expected results still need a specification or oracle; cannot reveal missing rules	Must be redone: new branch needs new test; refactoring changes the paths
C Fault-based	A fault catalogue (and mutation tool or knowledge of typical faults)	Moderately affected: the catalogue is built from past defects and rule text; gaps in the rules give gaps in the catalogue	Mutants are regenerated automatically; catalogue survives the change

1.6 Recommended combination for the 2-day window

1. **Day 1 morning – specification-based (A), about 1.5 tester-days.** Derive tests from every rule including limits; this covers the complaints (10 km, 11 pm, ₹180 with RIDE50, low wallet, cancel after start) and finds missing functionality.

2. **Day 1 afternoon – measure coverage (B as a measure, not as a design method).** Run coverage.py on the A suite; add structure-based tests only for uncovered branches. In Task 6 the black-box suite reached 100% statements and branches, so structural top-up was needed only for 2 surviving mutants.
3. **Day 2 – fault-based (C) top-up.** Generate mutants, add one test per surviving non-equivalent mutant, and finish with a regression pack.

Limits of this combination: it finds only faults that the specification, the code and the mutation operators can express. It will not find a rule that nobody wrote down, a fault that needs three or more interacting parameters, performance or concurrency failures, or errors shared by the test oracle and the code (both written from the same understanding). The ordering matters because the cheapest, most independent information (the specification) is used first and the code-dependent techniques are used only to fill measured gaps.

1.7 Critique: "an LLM can design all the test cases from the code"

Prompt given to the LLM: "Write unit tests for this Python function" followed by the text of Listing 1. The one-pass answer contained 17 tests (listed below).

#	Input	Expected (oracle)	Faults killed
1	0 km | w0 | h12 | – | 0 | CASH	INVALID	–
2	-5 km | w0 | h12 | – | 0 | CASH	INVALID	–
3	61 km | w0 | h12 | – | 0 | CASH	INVALID	–
4	1 km | w0 | h12 | – | 0 | CASH	50	–
5	2 km | w0 | h12 | – | 0 | CASH	50	–
6	5 km | w0 | h12 | – | 0 | CASH	86	–
7	10 km | w0 | h12 | – | 0 | CASH	146	–
8	20 km | w0 | h12 | – | 0 | CASH	246	–
9	5 km | w10 | h12 | – | 0 | CASH	96	–
10	5 km | w0 | h23 | – | 0 | CASH	107.5	F2
11	5 km | w0 | h2 | – | 0 | CASH	107.5	–
12	5 km | w0 | h12 | RIDE50 | 0 | CASH	86	–
13	20 km | w0 | h12 | RIDE50 | 0 | CASH	196	–
14	60 km | w30 | h23 | RIDE50 | 0 | CASH	800	F2, F5
15	5 km | w0 | h12 | – | 10 | WALLET	WALLET_LOW	–
16	5 km | w0 | h12 | – | 500 | WALLET	86	–
17	5 km | w0 | h12 | – | 500 | CASH	86	–

- **Which strategy it resembles:** mostly strategy B. It contains one test for each branch (invalid distance 0, −5, 61; slab 1, 2, 3; waiting; night 23 and 2; promo with and without discount; cap; wallet low and sufficient; cash). It has a few specification-style values (0, 2, 10 and hour 23), but not at the 0.1 km step on the other side (2.1, 10.1), not at 6 minutes, not at fare 199/200, not at wallet = fare.
- **Result:** 2 of 6 faults found (F2 and F5), which is no better than the hand-written structure-based suite, with 17 tests instead of 12.
- **What it misses:** the threshold values (F1, F3, F4, F6), all missing-functionality tests (RIDE50 once only; invalid wait/hour/mode), and any independent expected value: the LLM reads expected values from what the code computes, so a test generated from faulty code would pass on that faulty code.
- **Verdict:** the claim is false. An LLM is useful to draft the structural tests quickly (the 17 tests took seconds) but the specification-based and fault-based designs must still be added by a person who has read the rules.

> *Prediction recorded for Task 9: a suite built only from the code (strategy B or an LLM suite) will detect clearly fewer of ten seeded faults than a suite that combines specification-based and fault-based design.*

Task 2 – Random Testing: Uniform, Operational-Profile and Adaptive Random Generation

2.1 Set-up

Five faults were seeded in Listing 1 so that some fail only in a narrow region: R1 adds ₹6 only for 9.9 ≤ km ≤ 10.1 (wrong constant in a narrow band); R2 removes the night surcharge at hour 23 only (hour >= 23 → hour > 23); R3 moves the promo threshold from 200 to 250 (fails for fares 200–249); R4 lowers the cap from 800 to 700; R5 changes wallet < fare to wallet <= fare (fails only when balance equals the fare exactly). Each approach generated 40 tests per run; the expected fare comes from the independent oracle; a fault is "found" if at least one of the 40 tests shows a mismatch.

- **A Uniform:** km uniform on the 0.1 km grid 0.1–60, wait 0–30, hour 0–23, promo RIDE50 or none (50/50), wallet 0–1000, mode uniform over WALLET/UPI/CASH.
- **B Operational profile:** 70% rides 2.1–10 km, 20% above 10 km, 10% up to 2 km; 15% night rides (hours 23, 0–4); 5% promo use. Assumption added by the author: 30% of rides have a waiting time above 5 minutes (0–30), others 0–5; wallet and mode as in A.
- **C Adaptive random (fixed-size-candidate ART):** each new test is the best of 10 uniform candidates, where "best" means largest minimum Euclidean distance to the tests already chosen, with all six parameters normalised to 0–1.

2.2 Faults found with fixed seed 1 and over 10 seeds

Approach	Faults found, seed 1 (of 5)	Mean over 10 seeds	Std. dev.	Min – max	Count per seed (seeds 1–10)	Mean test number of first fault
A Uniform	3	2.2	0.98	0 – 3	3 1 3 2 2 3 0 3 3 2	7.7 (9 of 10 seeds)
B Operational	0	1.0	0.63	0 – 2	0 0 1 2 1 1 1 1 1 2	16.6 (8 of 10 seeds)
C Adaptive (ART)	3	2.7	0.78	1 – 4	3 3 2 2 3 3 4 1 3 3	8.2 (10 of 10 seeds)

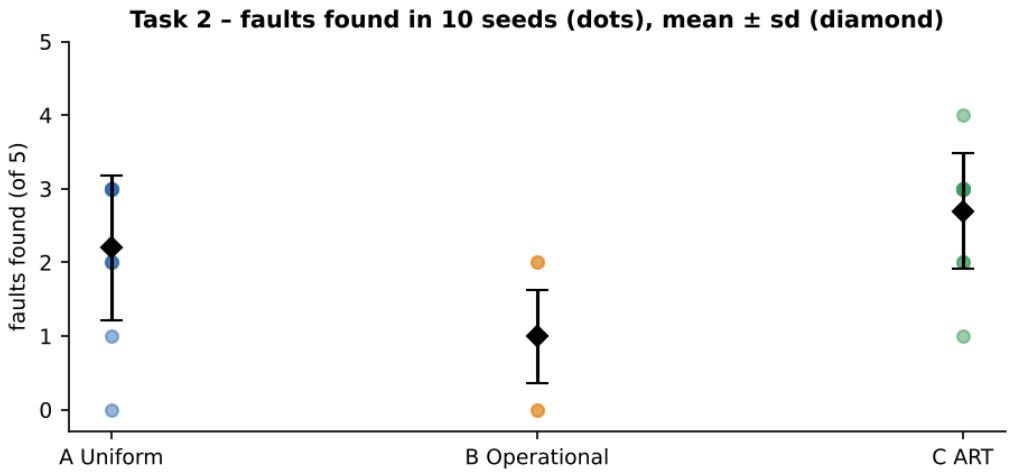


Figure 2.1 – Faults found per seed (dots) and mean ± standard deviation

Fault	A Uniform	B Operational	C Adaptive
R1 wrong constant only for 9.9<=km<=10.1	0 / 10	1 / 10	2 / 10
R2 no night surcharge at hour 23	7 / 10	6 / 10	9 / 10
R3 promo threshold 200 -> 250	9 / 10	0 / 10	7 / 10
R4 cap 800 -> 700	6 / 10	3 / 10	8 / 10
R5 wallet<fare -> wallet<=fare	0 / 10	0 / 10	1 / 10

Reading the tables: ART gave the highest mean (2.7 of 5 faults) and found at least one fault in all 10 seeds. Uniform random found 2.2 on average and beat ART only on R3 (9 seeds against 7), a fault whose region is wide (any fare from 200 to 249). The operational profile found the fewest (1.0 on average) because it deliberately sends most tests to the ordinary 2–10 km, daytime, no-promo ride where nothing is wrong; in 2 of 10 seeds it found nothing at all. R5 (balance exactly equal to the fare) was found by one ART run and by no uniform or profile run. The last column counts only the seeds in which at least one fault was found, and gives the average position of the first test that exposed any of the five faults.

2.3 Why uniform random inputs rarely hit boundary values

Quantity	Calculation	Value
P(km within 0.1 km of 2 or 10), continuous model	two windows of width 0.2 km in a range of 59.9 km: 0.4 / 59.9	0.668%
Same on the real 0.1 km grid	6 grid values (1.9, 2.0, 2.1, 9.9, 10.0, 10.1) out of 600	1.00%
Check by simulation (100,000 draws)	fraction of draws with |km−2| ≤ 0.1 or |km−10| ≤ 0.1	0.949%
P(at least one such test in 40 tests)	1 − (1 − 0.01)^40	33.1%
P(km is exactly 2.0 or 10.0)	2 / 600	0.33%
P(hour = 23) / P(wait = 5)	1/24 ; 1/31	4.17% ; 3.23%
P(a given 40-test run hits km = 10.0 exactly)	1 − (1 − 1/600)^40	6.5%

A uniform generator gives a boundary region of 0.1 km width only about 1% of the draws, so a 40-test run meets one of the four boundary values with probability one in three, and meets all of them (2.0, 2.1, 10.0, 10.1) essentially never. This is why the narrow fault R1 was found in 0 of 10 seeds by uniform random testing and in 2 of 10 by ART, which spreads points and therefore covers the 9.9–10.1 band slightly more often. Boundary faults are exactly the ones named in the passengers' complaints, so uniform random testing cannot replace designed boundary tests.

2.4 The oracle problem

For 40 random inputs nobody can calculate 40 fares by hand reliably, so an automatic oracle is needed. Here the oracle is a second implementation of the specification (spec_oracle: Decimal arithmetic, written from the rule table, about 20 lines, not copied from Listing 1). It evaluates one test in about 2 microseconds. Costs and limits of this approach:

- **Writing and checking the oracle costs more than the tests it feeds** (about one hour for these rules in the author's estimate, plus a review against the rule table). Every rule change must be made twice (code and oracle).
- **Common-mode failure:** if the developer and the tester misread the same rule (for example, whether the night period includes 4:59 am), the oracle and the code agree and the fault is invisible.
- **Rounding:** the oracle must use Decimal and round half up; comparing raw floats fails at the boundaries (Task 4 shows 96 of 3,600 fares differ from the exact value before rounding).
- **Partial oracles are cheaper:** properties such as "fare never exceeds 800" or "fare with promo is never higher than without" need no expected value, but detect only some faults (Task 10 uses this as metamorphic testing).

2.5 Where random testing is worth using in SwiftRide

- **Worth using:** (a) as a cheap robustness and crash check on the whole input space (any exception or non-numeric result is a failure without an oracle); (b) as a regression complement after a rule change, because ART found at least one fault in 10 of 10 seeds with only 40 tests; (c) for reliability estimation, but only with the operational profile (2.6).
- **Must be replaced by designed tests:** limits and thresholds (2, 10, 60 km; 5 min; hour 23/4/5; fare 200; cap 800; balance = fare) – R1 and R5 were missed by uniform random in all 10 seeds; and rule interactions that need a specific combination (promo with fare ≥ 200 and night).
- **Decision:** 40 ART tests are added to the nightly run only; the release criterion stays with designed tests.

2.6 Judging “random testing gives an unbiased reliability estimate only if inputs follow the operational profile”

Quantity (all five faults treated as one faulty release)	Value
True failure probability under the operational profile (100,000 draws)	4.42%
True failure probability under the uniform distribution (100,000 draws)	10.03%
Mean estimate from 10 runs of 40 operational-profile tests (approach B)	3.75%
Mean estimate from 10 runs of 40 uniform tests (approach A)	8.75%

The estimate from approach B (3.75%) is close to the failure probability that real riders would experience (4.42%); the difference is inside the sampling error of 400 tests (about ±1 percentage point). Approach A estimates 8.75%, nearly double, because uniform generation gives 50% promo rides, long rides and night hours far more often than real traffic. The same table shows per fault how the profile matters (for example R3, promo threshold: 3.62% of uniform inputs fail but only 0.12% of operational inputs). The statement is therefore supported: random testing estimates reliability correctly only when the sampling distribution is the operational profile. Two conditions are hidden in it: the profile itself must be measured from real data (the 70/20/10 split here is the one given in the assignment), and the faults must be failures that matter to users, since a profile-based estimate says nothing about a rare but expensive boundary case.

> *Prediction recorded for Task 9: a 54-test uniform random suite will detect about half of ten seeded faults and will not reliably reach boundary faults.*

Task 3 – Equivalence Class Partitioning: Valid-Class, Robust and Classification-Tree Variants

3.1 Input conditions and equivalence classes

Every parameter was partitioned from the specification. For the hour, the night period is split into two classes (hour 23 and hours 0–4) because Listing 1 tests it with an OR of two different conditions (hour >= 23 and hour < 5, line 12); one class that merged them would let a test cover only one side of the OR. The promo code has three valid classes because the program treats “no code” and “another code” identically, but a tester cannot know that without trying both.

ID	Parameter	Class	V / I	Representative value
km1	km	0 < km <= 2	Valid	1.5
km2	km	2 < km <= 10	Valid	6
km3	km	10 < km <= 60	Valid	20
km4	km	km <= 0	Invalid	0
km5	km	km > 60	Invalid	61
w1	wait	0 <= wait <= 5 (free)	Valid	3
w2	wait	5 < wait <= 30 (charged)	Valid	12
w3	wait	wait < 0	Invalid	-1
w4	wait	wait > 30	Invalid	31
h1	hour	5 <= hour <= 22 (day)	Valid	12
h2	hour	hour = 23 (night, late)	Valid	23
h3	hour	0 <= hour <= 4 (night, early)	Valid	2
h4	hour	hour < 0	Invalid	-1
h5	hour	hour > 23	Invalid	24
p1	promo	"RIDE50"	Valid	'RIDE50'
p2	promo	no code	Valid	''
p3	promo	other code	Valid	'SAVE10'
v1	wallet	wallet >= fare	Valid	1000
v2	wallet	wallet < fare	Valid	10
v3	wallet	wallet < 0	Invalid	-5
m1	mode	WALLET	Valid	'WALLET'
m2	mode	UPI	Valid	'UPI'
m3	mode	CASH	Valid	'CASH'
m4	mode	other mode	Invalid	'BITCOIN'

Totals: 24 classes = 16 valid + 8 invalid (km 5, wait 4, hour 5, promo 3, wallet 3, mode 4).

3.2 Test cases by the three variants

Variant A – valid-class partitioning (several classes per test)

Minimum number of tests = largest number of classes of one parameter (3 for km, hour, promo, mode), so 3 tests cover all 16 valid classes. The table shows the classes each test covers and the seeded faults it kills.

#	Input	Expected	Classes covered	Faults killed
1	1.5 km | w3 | h12 | RIDE50 | 1000 | WALLET	50	km1, w1, h1, p1, v1, m1	–
2	6 km | w12 | h23 | – | 10 | UPI	140	km2, w2, h2, p2, v2, m2	E1, E3, E4, E6
3	20 km | w3 | h2 | SAVE10 | 1000 | CASH	307.5	km3, w1, h3, p3, v1, m3	E2, E4

Variant B – robust partitioning (A plus one separate test per invalid class)

Each invalid class is tested alone so that a rejection caused by one bad value cannot hide the handling of another. Tests 4–11 are the invalid-class tests. For these the expected value is INVALID (strict oracle).

#	Input	Expected	Classes covered	Faults killed
1	1.5 km | w3 | h12 | RIDE50 | 1000 | WALLET	50	km1, w1, h1, p1, v1, m1	–
2	6 km | w12 | h23 | – | 10 | UPI	140	km2, w2, h2, p2, v2, m2	E1, E3, E4, E6
3	20 km | w3 | h2 | SAVE10 | 1000 | CASH	307.5	km3, w1, h3, p3, v1, m3	E2, E4
4	0 km | w3 | h12 | – | 1000 | CASH	INVALID	km4	–
5	61 km | w3 | h12 | – | 1000 | CASH	INVALID	km5	–
6	6 km | w-1 | h12 | – | 1000 | CASH	98	w3	E1
7	6 km | w31 | h12 | – | 1000 | CASH	150	w4	E1, E3
8	6 km | w3 | h-1 | – | 1000 | CASH	122.5	h4	E1, E4
9	6 km | w3 | h24 | – | 1000 | CASH	122.5	h5	E1, E4
10	6 km | w3 | h12 | – | -5 | WALLET	WALLET_LOW	v3	–
11	6 km | w3 | h12 | – | 1000 | BITCOIN	98	m4	E1

Variant C – classification tree (distance × time of day × promo, with wait and payment rotated)

Classification tree: root “ride” → classifications Distance {km1, km2, km3} × Time {h1, h2, h3} × Promo {p1, p2}. All 18 leaf combinations are tests; the remaining classifications (wait, payment: WALLET enough / WALLET low / UPI / CASH) are rotated through the combinations so that each appears at least 4 times. The 8 invalid-class tests of variant B are appended.

#	Input	Expected	Classes covered	Faults killed
1	1.5 km | w3 | h12 | RIDE50 | 1000 | WALLET	50	km1, h1, p1, w1, m1, v1	–
2	1.5 km | w12 | h12 | – | 10 | WALLET	WALLET_LOW	km1, h1, p2, w2, m1, v2	–
3	1.5 km | w3 | h23 | RIDE50 | 1000 | UPI	62.5	km1, h2, p1, w1, m2, v1	E4
4	1.5 km | w12 | h23 | – | 1000 | CASH	80	km1, h2, p2, w2, m3, v1	E3, E4
5	1.5 km | w3 | h2 | RIDE50 | 1000 | WALLET	62.5	km1, h3, p1, w1, m1, v1	E4
6	1.5 km | w12 | h2 | – | 10 | WALLET	WALLET_LOW	km1, h3, p2, w2, m1, v2	–
7	6 km | w3 | h12 | RIDE50 | 1000 | UPI	98	km2, h1, p1, w1, m2, v1	E1
8	6 km | w12 | h12 | – | 1000 | CASH	112	km2, h1, p2, w2, m3, v1	E1, E3
9	6 km | w3 | h23 | RIDE50 | 1000 | WALLET	122.5	km2, h2, p1, w1, m1, v1	E1, E4
10	6 km | w12 | h23 | – | 10 | WALLET	WALLET_LOW	km2, h2, p2, w2, m1, v2	–
11	6 km | w3 | h2 | RIDE50 | 1000 | UPI	122.5	km2, h3, p1, w1, m2, v1	E1, E4
12	6 km | w12 | h2 | – | 1000 | CASH	140	km2, h3, p2, w2, m3, v1	E1, E3, E4
13	20 km | w3 | h12 | RIDE50 | 1000 | WALLET	196	km3, h1, p1, w1, m1, v1	E2, E5
14	20 km | w12 | h12 | – | 10 | WALLET	WALLET_LOW	km3, h1, p2, w2, m1, v2	–
15	20 km | w3 | h23 | RIDE50 | 1000 | UPI	257.5	km3, h2, p1, w1, m2, v1	E2, E4, E5, E7
16	20 km | w12 | h23 | – | 1000 | CASH	325	km3, h2, p2, w2, m3, v1	E2, E3, E4
17	20 km | w3 | h2 | RIDE50 | 1000 | WALLET	257.5	km3, h3, p1, w1, m1, v1	E2, E4, E5, E7
18	20 km | w12 | h2 | – | 10 | WALLET	WALLET_LOW	km3, h3, p2, w2, m1, v2	–
19	0 km | w3 | h12 | – | 1000 | CASH	INVALID	km4	–
20	61 km | w3 | h12 | – | 1000 | CASH	INVALID	km5	–

#	Input	Expected	Classes covered	Faults killed
21	6 km | w-1 | h12 | – | 1000 | CASH	98	w3	E1
22	6 km | w31 | h12 | – | 1000 | CASH	150	w4	E1, E3
23	6 km | w3 | h-1 | – | 1000 | CASH	122.5	h4	E1, E4
24	6 km | w3 | h24 | – | 1000 | CASH	122.5	h5	E1, E4
25	6 km | w3 | h12 | – | -5 | WALLET	WALLET_LOW	v3	–
26	6 km | w3 | h12 | – | 1000 | BITCOIN	98	m4	E1

Variant	Tests	Classes covered	Seeded faults found
A Valid-class	3	16 / 24	5 / 7
B Robust	11	24 / 24	5 / 7
C Classification tree	26	23 / 24	6 / 7

3.3 Seven seeded faults and faults found per variant

Fault	A (3 tests)	B (11 tests)	C (26 tests)
E1 slab-2 rate 12 -> 11	found by test 2	found by test 2	found by test 7
E2 slab-3 rate 10 -> 9	found by test 3	found by test 3	found by test 13
E3 waiting rate 2 -> 3	found by test 2	found by test 2	found by test 4
E4 night factor 1.25 -> 1.2	found by test 2	found by test 2	found by test 3
E5 promo amount 50 -> 40	MISSED	MISSED	found by test 13
E6 wallet check for all modes	found by test 2	found by test 2	MISSED
E7 promo applied before night factor	MISSED	MISSED	found by test 15

- **E5 (promo amount 50 → 40) is missed by A and B** because their tests never combine a promo code with a fare of at least 200; the promo is accepted but not applied (fares 50 and 86). Only the classification tree, which crosses RIDE50 with km3, applies the discount.
- **E7 (promo applied before the night factor) is missed by A and B** for the same reason: the order of two operations only matters when both fire in one ride.
- **E6 (wallet check made for every mode) is missed by C** although C is the largest suite: its payment rotation used a sufficient balance for UPI and CASH, so a low balance with a non-wallet mode never occurred. A had that combination by luck in test 2 (UPI, wallet 10). Larger suites are not automatically better.

3.4 Results the invalid classes expose in Listing 1 (robustness defects)

Class	Meaning	Input	Listing 1 returns	Expected	Verdict
km4	km ≤ 0	0 km | w3 | h12 | – | 1000 | CASH	INVALID	INVALID	ok
km5	km > 60	61 km | w3 | h12 | – | 1000 | CASH	INVALID	INVALID	ok
w3	wait < 0	6 km | w-1 | h12 | – | 1000 | CASH	98	INVALID	DEFECT
w4	wait > 30	6 km | w31 | h12 | – | 1000 | CASH	150	INVALID	DEFECT
h4	hour < 0	6 km | w3 | h-1 | – | 1000 | CASH	122.5	INVALID	DEFECT
h5	hour > 23	6 km | w3 | h24 | – | 1000 | CASH	122.5	INVALID	DEFECT
v3	wallet < 0	6 km | w3 | h12 | – | -5 | WALLET	WALLET_LOW	INVALID	DEFECT
m4	mode = BITCOIN	6 km | w3 | h12 | – | 1000 | BITCOIN	98	INVALID	DEFECT

Six of the eight invalid classes are accepted silently; for example hour 24 is charged as a night ride and a negative wait is charged as a normal ride. These are not seeded faults; they are missing validation in the code that a valid-class suite (A) can never reveal. This is the practical argument for robust partitioning.

3.5 When the equivalence assumption breaks

Equivalence partitioning assumes that all members of a class are processed alike. Class km2 (2 < km ≤ 10) breaks this assumption because fare ≥ 200 – the condition for the promo – is reachable inside the class only when waiting time or the night factor is added. Measured with promo RIDE50 and mode CASH:

Distance (all in class km2)	Wait	Hour	Fare without promo	Fare with RIDE50	Effect
10 km	0	12	146	146	no discount
10 km	30	12	196	196	no discount
10 km	0	23	182.5	182.5	no discount
10 km	30	23	245	195	promo discount ₹50
2.1 km	30	23	126.5	126.5	no discount
9.9 km	30	23	243.5	193.5	promo discount ₹50

Three members of the same class km2 (10 km at noon, 10 km with wait 30 at noon, 10 km at 23:00) get no discount, while the same distance with wait 30 at 23:00 (fare ₹245) gets ₹50 off. The class must be split: km2a “2–10 km, fare < 200 whatever the extras” and km2b “2–10 km where wait and night can lift the fare to ≥ 200”. The rule for splitting a class that is too coarse: split whenever (1) a threshold on a derived value (fare, total, running sum) lies inside the class, (2) two members trigger different branches when combined with the same values of the other parameters, or (3) a seeded or real fault is found inside a class by one member and not by another.

3.6 Evaluation and recommendation

Variant	Tests	Faults found (of 7)	Invalid classes tested	Effort (estimate)	Main weakness
A Valid-class	3	5	0 of 8	20 min	misses interactions and all invalid inputs
B Robust	11	5	8 of 8	35 min	misses interactions (E5, E7)
C Classification tree	26	6	8 of 8 (appended)	60 min	large; missed E6 through rotation

Recommended: variant B as the base, plus the interaction tests of variant C for the combination slab × night × promo only (about 6 extra tests, not all 18). B is the cheapest variant that exposes the six validation gaps; C’s extra tests are valuable only where a threshold depends on a combination, which is exactly the fare ≥ 200 rule. The recommendation is not “C because it is biggest”: C missed a fault that the smallest suite caught.

3.7 Critique: equivalence partitioning by an LLM

Prompt: “Partition the input parameters of calculate_fare(km, wait, hour, promo, wallet, mode) into equivalence classes.” One-pass LLM answer: km {≤0, 0.1–2, 2–10, 10–60, >60}; wait {0–5, 6–30}; hour {night 0–4 and 23, day 5–22}; promo {RIDE50, other}; wallet {sufficient, insufficient}; mode {WALLET, UPI, CASH} = 16 classes.

Item	Author’s partition	LLM partition	Consequence
Number of classes	24	16	8 fewer
Missing invalid classes	wait<0, wait>30, hour<0, hour>23, wallet<0, mode other (6)	none listed	six validation defects of Listing 1 would not be tested
Wrongly merged	hour 23 and hours 0–4 are two classes (two conditions of one OR)	merged into one “night” class	one test may cover only one side of the OR; fault F2/BF1 (hour > 23) needs 23 specifically
Merged	promo: “no code” and “other code” separate	merged as “other”	acceptable here, but it hides whether an unknown code is rejected
Interaction	km2 flagged as too coarse	not mentioned	promo/night/wait interaction missed (E5, E7)

The LLM answer is a good first draft of the valid classes (all 16 valid classes of the author's table are covered) but it contains no invalid classes beyond the distance and no sign that a class may need splitting. As in Task 1 it must be completed by a person who reads the specification.

> *Prediction recorded for Task 9: equivalence-class tests alone (variant B) will miss the interaction fault "promo applied before night factor".*

Task 4 – Boundary Value Analysis: Two-Value, Robust and Worst-Case Boundary Testing

4.1 Boundaries from the specification

Parameter	Boundary	Value just on the lower side	Value just on the upper side	Precision / note
km	lower limit of valid range	0 (INVALID)	0.1 (valid, ₹50)	0.1 km; valid range 0.1–60
km	slab 1 / slab 2	2.0 (₹50)	2.1 (₹51.20)	0.1 km; slabs are continuous at 2
km	slab 2 / slab 3	10.0 (₹146)	10.1 (₹147)	0.1 km; continuous at 10
km	upper limit of valid range	60.0 (valid)	60.1 (INVALID)	0.1 km
wait	free / charged	5 (₹0 extra)	6 (₹2 extra)	whole minutes
wait	range limits	−1 (invalid) | 0	30 | 31 (invalid)	whole minutes
hour	night ends	4 (night)	5 (day)	hour 4:59 am is still night
hour	night starts	22 (day)	23 (night)	hour 23 = 11 pm
hour	range limits	−1 (invalid) | 0	23 | 24 (invalid)	integers 0–23

4.2 Test sets of the three methods

Method A (two-value): for every boundary the value on the boundary and the nearest value on the other side, other parameters at a neutral value (km 6, wait 0, hour 12, no promo, cash). Method B (robust): min−, min, min+, nominal, max−, max, max+ for each parameter with the other two at nominal (km 30, wait 15, hour 12). Method C (worst case): the five values min, min+, nominal, max−, max of every parameter in all combinations.

Parameter	A: two-value values	B: robust values	C: five values
km	0, 0.1, 2, 2.1, 10, 10.1, 60, 60.1 (8)	0, 0.1, 0.2, 30, 59.9, 60, 60.1 (7)	0.1, 0.2, 30, 59.9, 60 (5)
wait	−1, 0, 5, 6, 30, 31 (6)	−1, 0, 1, 15, 29, 30, 31 (7)	0, 1, 15, 29, 30 (5)
hour	−1, 0, 4, 5, 22, 23, 24 (7 distinct)	−1, 0, 1, 12, 22, 23, 24 (7)	0, 1, 12, 22, 23 (5)

Method	Rule	Tests for km, wait, hour together	Tests (n = 3 parameters)
A Two-value	2 per boundary	8 + 6 + 8 − 1 duplicate (hour 23)	21
B Robust	7 per parameter, others nominal	7 + 7 + 7 − 2 duplicates of the all-nominal test	19
C Worst case	5^n	5 × 5 × 5	125 (a robust worst case would be 7^3 = 343)

4.3 Six boundary faults and results

ID	Line	Fault	A (21 tests)	B (19 tests)	C (125 tests)
BF1	12	hour>=23 -> hour>23	found (test 20)	found (test 18)	found (test 5)
BF2	6	km<=10 -> km<=10.1	found (test 6)	MISSED	MISSED
BF3	10	wait>5 -> wait>6	found (test 12)	MISSED	MISSED
BF4	2	km>60 -> km>=60	found (test 7)	found (test 6)	found (test 101)
BF5	12	hour<5 -> hour<4	found (test 17)	MISSED	MISSED
BF6	2	km<=0 -> km<0	found (test 1)	found (test 1)	MISSED

Method	Tests	Faults found (of 6)	Faults per 10 tests
A Two-value	21	6	2.86

Method	Tests	Faults found (of 6)	Faults per 10 tests
B Robust	19	3	1.58
C Worst case	125	2	0.16

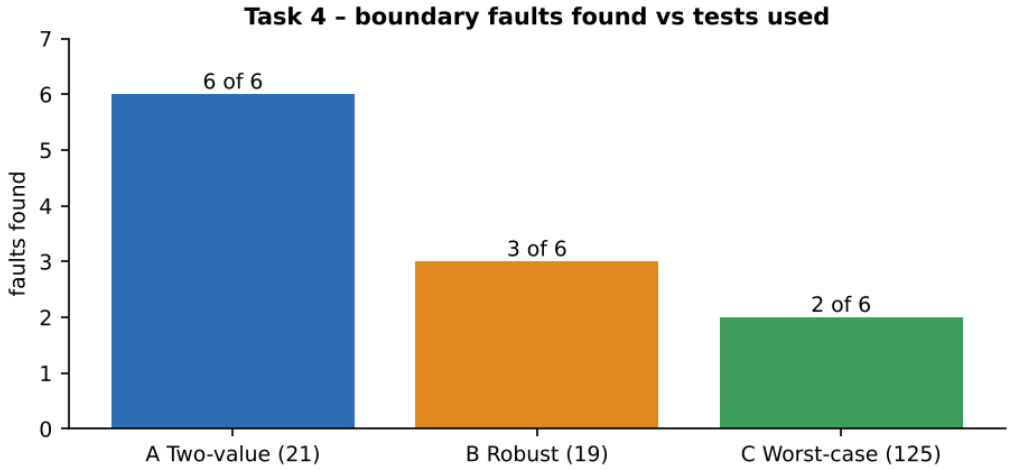


Figure 4.1 – Boundary faults found against number of tests

Method A finds all six; B and C, which have more values per parameter, find only 3 and 2. The reason is visible in the value lists: robust and worst-case testing take only the **external limits** of each range (min, max) and a nominal value. They never contain 2.1, 10.1, hour 22/23 pairs at the internal limits or wait 5/6, which are the internal boundaries that decide the rate. Worst-case testing multiplies 125 combinations but every combination still uses the same blind values: it spends 125 tests to find one fault fewer than A finds with 21. Counting boundaries, not values per range, is the lesson.

4.4 Floating-point problems at boundaries

Distances are floating-point numbers, and 0.1 cannot be represented exactly in binary. Computed in Python:

```
>>> 2.1 - 2
0.10000000000000009
>>> (2.1 - 2) * 12
1.200000000000001
>>> 10.1 - 10
0.09999999999999964
>>> (10.1 - 10) * 10
0.9999999999999964
```

km	Raw value computed by the formula	After round(x, 2)	Exact decimal value
2.1	50 + (2.1−2)×12 = 51.2 (hidden by repr)	51.2	51.20
5.3	89.6	89.6	89.60
10.1	146 + (10.1−10)×10 = 147.0 (hidden by repr)	147.0	147.00
2.2 (wait 0, noon)	52.400000000000006	52.4	52.40
2.3 (wait 0, noon)	53.599999999999994	53.6	53.60

Measured over 3,600 fares (600 distances × hour 12 and 23 × wait 0, 12, 30): 96 fares (2.7%) are not exactly equal to the exact decimal value before rounding, and 0 differ after rounding to 2 decimals. A test written as assert fare == 52.4 would fail on 52.400000000000006; the oracle must therefore round (or use pytest.approx / Decimal) and compare the rounded values. Floating-point error can also flip a decision such as fare >= 200; on the same grid 0 threshold decisions flipped, but the test for the exact threshold was built with integer inputs (km 14, wait 12 → 146 + 40 + 14 = 200 exactly) to avoid relying on luck: km 15.4 gives 200.0 only by rounding of the product 54.00000000000001.

4.4a Equivalent boundary mutants – a finding of the analysis

Because the fare is continuous at 2 km and 10 km, the literal off-by-one edits km <= 2 → km < 2 and km <= 10 → km < 10 cannot change any output (proved on 4,303,728 inputs in Section 0.3). Boundary value analysis therefore cannot find "km <= 10 written as km < 10", and no test can. A boundary fault at a slab limit is detectable only when the slab limit is shifted (10.1 instead of 10) or when a slab constant is wrong. The same holds for wait 5 vs 6 charging 0 extra.

4.5 Evaluation – best balance for the 2-day window

Method	Tests	Faults found	Run time (in-process)	Verdict
A Two-value	21	6 / 6	< 1 ms	best balance
B Robust	19	3 / 6	< 1 ms	misses internal limits
C Worst case	125	2 / 6	< 5 ms	most tests, fewest faults

Method A is chosen. Worst-case boundary testing is justified only when (a) the parameters interact at their limits – for example cap 800 together with night and the maximum wait, or fare exactly 200 with night; (b) the list of values per parameter already contains the internal boundaries (7 or more values per parameter), and (c) tests are cheap (here 125 tests would take less than one second to run). For SwiftRide it is restricted to the three parameters that feed the promo and cap decisions.

4.6 Would boundary value analysis alone have prevented the passenger complaints?

Complaint	Fault that explains it	Found by BVA method A?	What else is needed
"No night surcharge at exactly 11 pm"	hour >= 23 → hour > 23 (BF1)	Yes – hour 23 is in A, B and C	a rule review: was the intended night start 22:00 or 23:00?
"10 km ride on the wrong slab"	An off-by-one at 10 km is equivalent (no effect); a wrong constant (146 → 140) or rate is the real cause	146 → 140: yes (A, B, C). Slab-2 constant or rate (50 → 48, 12 → 11): only A	tests inside every slab (not only at the limits); a continuity check "fare(10.0) = fare(10.1) − 1.0"
Promo RIDE50 on ₹180	threshold fault (200 → 150) – an interaction of promo and fare	No: promo is not a BVA parameter	decision-table test with fare 196/200 (Task 5)
Wallet payment with low balance	wallet <= fare or missing check	Not by km/wait/hour	boundary on the balance (balance = fare) and decision-table test
Cancel after ride started	missing state rule	No	state-transition test (Task 5)

BVA explains and prevents the two limit complaints only when the limit is an input of the function and the oracle is correct. The other three complaints are caused by rules on combinations or sequences, which need decision-table and state-transition testing.

> *Prediction recorded for Task 9: the two-value boundary suite will find all six boundary-type faults among the ten seeded faults; the pairwise suite alone will not.*

Task 5 – Decision Table, State-Transition and Pairwise Testing

5.1 Decision table for the promo, cap and payment rules

Conditions: C1 km > 2; C2 night ride; C3 promo = RIDE50; C4 fare after waiting and night ≥ ₹200; C5 payment mode = WALLET; C6 wallet balance ≥ fare. Actions: A1 slab fare (flat ₹50 or ₹50 + ₹12/km …); A2 ×1.25; A3 promo −₹50; A4 cap to ₹800; A5 return WALLET_LOW; A6 return the fare.

Step	Rules	Reason
Full table	64 = 2^6	all combinations of six Y/N conditions
Remove don't-care entries	36	C4 is irrelevant when C3 = N (no promo code, so the fare level does not matter); C6 is irrelevant when C5 = N (no balance check outside WALLET)
Remove impossible rules	30	C1 = N (km ≤ 2) with C4 = Y is impossible: the largest fare for km ≤ 2 is (50 + 50 for 30 min waiting) × 1.25 = ₹125 < ₹200
Reduce by independence of C1, C2	9 core rules + 3 data rules	C1 and C2 only change the amount (A1, A2), not which rule of the promo × payment matrix fires; one rule of each pair is chosen so that both values of C1 and C2 occur in the core. Cap is tested with 3 extra rules (R10–R12)

Rule	C1	C2	C3	C4	C5	C6	A3 promo	A4 cap	Result	Output
R1	N	N	N	N	N	–	–	–	fare	50
R2	Y	Y	N	N	Y	Y	–	–	fare	122.5
R3	Y	N	N	N	Y	N	–	–	WALLET_LOW	WALLET_LOW
R4	Y	N	Y	Y	N	–	Y	–	fare	196
R5	Y	N	Y	Y	Y	Y	Y	–	fare	196
R6	Y	N	Y	Y	Y	N	Y	–	WALLET_LOW	WALLET_LOW
R7	Y	N	Y	N	N	–	–	–	fare	98
R8	Y	N	Y	N	Y	Y	–	–	fare	196
R9	Y	N	Y	N	Y	N	–	–	WALLET_LOW	WALLET_LOW
R10	Y	N	Y	Y	N	–	Y	–	fare	150
R11	Y	Y	N	Y	Y	Y	–	Y	fare	800
R12	Y	Y	N	Y	Y	Y	–	Y	fare	800

Rule	Test input	Description
R1	1.5 km | w0 | h12 | – | 0 | CASH	no promo, non-wallet
R2	6 km | w0 | h23 | – | 500 | WALLET	no promo, wallet ok, night
R3	6 km | w0 | h12 | – | 10 | WALLET	no promo, wallet low
R4	20 km | w0 | h12 | RIDE50 | 0 | UPI	promo applied (fare>=200), non-wallet
R5	20 km | w0 | h12 | RIDE50 | 500 | WALLET	promo applied, wallet ok
R6	20 km | w0 | h12 | RIDE50 | 150 | WALLET	promo applied, wallet low (196 > 150)
R7	6 km | w0 | h12 | RIDE50 | 0 | CASH	promo not applied (fare<200), non-wallet
R8	15 km | w0 | h12 | RIDE50 | 500 | WALLET	promo not applied (196), wallet ok
R9	6 km | w0 | h12 | RIDE50 | 10 | WALLET	promo not applied, wallet low
R10	14 km | w12 | h12 | RIDE50 | 500 | UPI	fare exactly 200: promo applied
R11	60 km | w30 | h23 | – | 900 | WALLET	cap: fare capped at 800, wallet 900
R12	60 km | w30 | h23 | – | 800 | WALLET	cap: wallet = 800 accepted

5.2 State-transition model of the ride life cycle

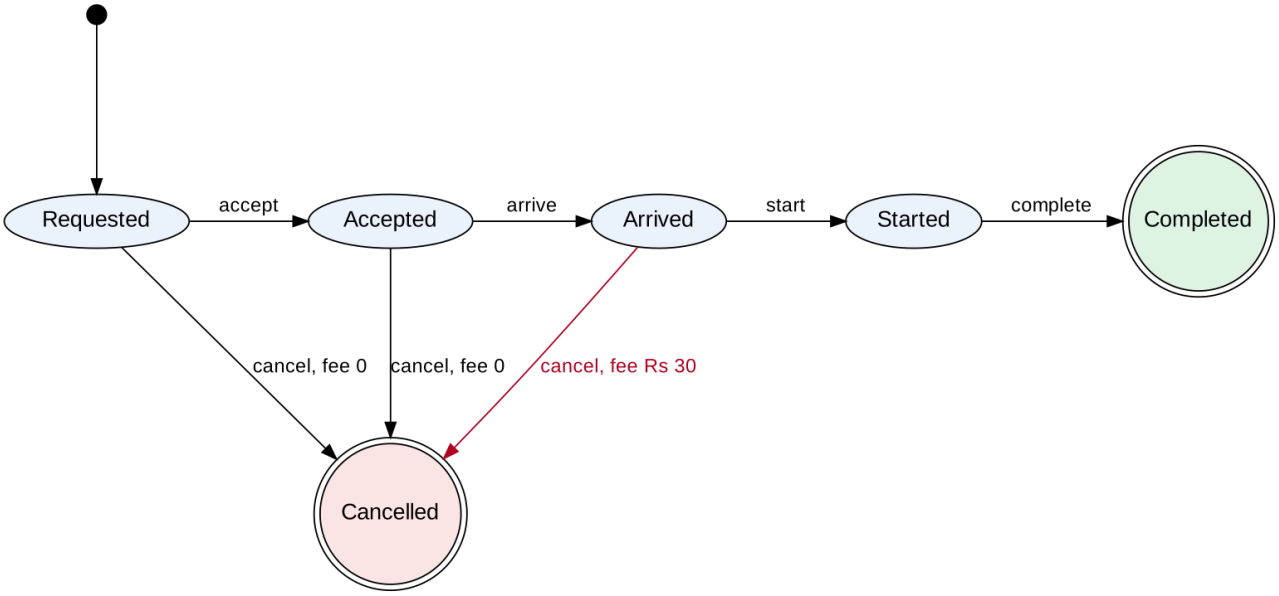


Figure 5.1 – Ride life cycle (states, events, cancellation fee on the Arrived → Cancelled transition)

State table (rows = current state, columns = event; "–" = event not allowed, the ride must stay in its state and report an error):

State	accept	arrive	start	complete	cancel
Requested	Accepted	–	–	–	Cancelled (fee 0)
Accepted	–	Arrived	–	–	Cancelled (fee 0)
Arrived	–	–	Started	–	Cancelled (fee ₹30)
Started	–	–	–	Completed	–
Completed	–	–	–	–	–
Cancelled	–	–	–	–	–

Cells: 30 (6 states × 5 events). Valid transitions: 7. Invalid transitions: 23, of which 8 were chosen as tests (the most likely programming mistakes and all the complaint scenarios).

Test	Kind	Event sequence (bold = event under test)	Expectation
ST1	valid	(new ride) → **accept**	from Requested: expect Accepted
ST2	valid	accept → **arrive**	from Accepted: expect Arrived
ST3	valid	accept → arrive → **start**	from Arrived: expect Started
ST4	valid	accept → arrive → start → **complete**	from Started: expect Completed
ST5	valid	(new ride) → **cancel**	from Requested: expect Cancelled
ST6	valid	accept → **cancel**	from Accepted: expect Cancelled
ST7	valid	accept → arrive → **cancel**	from Arrived: expect Cancelled
ST8	invalid	accept → arrive → start → **cancel**	from Started: expect rejected
ST9	invalid	accept → arrive → **complete**	from Arrived: expect rejected
ST10	invalid	(new ride) → **start**	from Requested: expect rejected
ST11	invalid	accept → **complete**	from Accepted: expect rejected
ST12	invalid	accept → arrive → start → complete → **cancel**	from Completed: expect rejected
ST13	invalid	cancel → **accept**	from Cancelled: expect rejected
ST14	invalid	(new ride) → **arrive**	from Requested: expect rejected
ST15	invalid	accept → **start**	from Accepted: expect rejected

Coverage: all 7 valid transitions (0-switch coverage) and 8 invalid transitions (cancel after Started, complete from Arrived, start from Requested, complete from Accepted, cancel after Completed, accept after Cancelled,

arrive from Requested, start from Accepted). The cancellation fee was checked as well: cancel at Arrived → fee ₹30, cancel at Accepted → fee ₹0.

5.3 Pairwise suite

Parameters and values: km class {≤2 (1.5), 2–10 (6), >10 (20)}; wait class {≤5 (3), >5 (12)}; hour class {day (12), 23, early (2)}; promo {RIDE50, none}; wallet {sufficient 1000, low 10}; mode {WALLET, UPI, CASH}. The suite was generated with the allpairspy library.

Quantity	Value
Full Cartesian product	3×2×3×2×2×3 = 216 tests
Pairwise suite	11 tests (5.1% of the product)
Parameter-value pairs to cover	93
Pairs covered by the suite	93 (100%)
Reduction	20×

#	km	wait	hour	promo	wallet	mode	Expected
1	1.5	3	12	RIDE50	1000	WALLET	50
2	6	12	23	–	10	WALLET	WALLET_LOW
3	20	12	2	RIDE50	10	UPI	275
4	20	3	23	–	1000	CASH	307.5
5	6	3	2	RIDE50	1000	CASH	122.5
6	1.5	12	12	–	10	CASH	64
7	1.5	3	2	–	10	UPI	62.5
8	6	12	12	–	1000	UPI	112
9	20	12	12	–	1000	WALLET	260
10	1.5	12	23	RIDE50	1000	UPI	80
11	1.5	12	2	–	1000	WALLET	80

5.4 Which technique finds which kind of fault

Two faults were seeded in the rule logic of Listing 1 and two in the ride life-cycle implementation (ride.py). RL1: promo condition “and fare >= 200” → “or fare >= 200” (the promo is applied to any ride with a fare of at least ₹200 even without the code). RL2: the wallet check moved before the cap (the balance is compared with the uncapped fare). LF1: cancel allowed after Started. LF2: complete allowed directly from Arrived. A third life-cycle fault LF3 (cancellation fee charged already at Accepted) was added for the fee rule.

Fault	A Decision table (12 tests)	B State transition (15 tests)	C Pairwise (11 tests)
RL1 promo: “and fare>=200” → “or fare>=200”	FOUND	not applicable	FOUND
RL2 wallet check before cap	FOUND	not applicable	MISSED
LF1 cancel after Started	not applicable	FOUND	not applicable
LF2 complete from Arrived	not applicable	FOUND	not applicable
LF3 fee charged at Accepted	not applicable	FOUND	not applicable

The decision table finds both rule-logic faults, including RL2, which needs a capped fare with a wallet balance of exactly ₹800 (rule R12). Pairwise found RL1 but missed RL2: the combination “wallet = fare after cap” is a boundary on a derived value, not a pair of parameter classes. State-transition testing found all three life-cycle faults in tests ST8, ST9 and ST6 respectively and cannot see the fare rules at all. Each technique finds the fault class it models; none finds all.

5.5 Evaluation and recommendation

Criterion	A Decision table	B State transition	C Pairwise
Tests	12 (30 rules reduced)	15 (7 valid + 8 invalid)	11 (of 216)
Faults found of 5 (rule + life cycle)	2 of 2 rule faults	3 of 3 life-cycle faults	1 of 2 rule faults
Effort (estimate)	60 min	45 min	20 min with a tool
Maintenance	a new rule adds a column; table grows with conditions	a new state or event adds a row/column	regenerate from the parameter list; no hand work
Best use in SwiftRide	Fare and Promo, Payment and Wallet rules (Listing 1)	Ride Management module	Booking/UI parameter combinations and the new parameters of Task 10

Recommendation: use all three, each in its module, because they find different faults: decision table for the fare and payment rules, state-transition for the ride life cycle, pairwise where there are many independent parameters and no business rule links them.

5.6 Critique: decision table and state diagram from an LLM

Prompt: “Create a decision table for the promo, cap and payment rules and a state diagram for the ride life cycle of SwiftRide.” The one-pass answer contained a decision table with five conditions (km > 2, night, promo RIDE50, fare ≥ 200, wallet sufficient) listed as the full 2^5 = 32 rules, and a state diagram with six states (including Cancelled) and seven transitions.

Item	LLM draft	Author’s own	Finding
Number of rules	32 (all combinations)	30 feasible after reduction (6 conditions)	the draft is not reduced
Impossible rules	8 of the 32 rules have km ≤ 2 and fare ≥ 200	removed (max fare for km ≤ 2 is ₹125)	8 impossible test cases would be produced; they cannot be executed
Missing rules	payment mode is folded into “wallet sufficient”; the cap (₹800) has no condition or action	mode WALLET/non-WALLET separate; cap rules R11–R12	cap and non-wallet modes missing
State diagram	same 7 valid transitions	same	no impossible transitions found in this run
Cancellation fee	mentioned in text only	on the Arrived → Cancelled edge (₹30)	fee rule not shown in the diagram; no test
Invalid transitions	not listed	8 tests	the draft gives no negative tests

For the state diagram this run found no impossible transition and no missing valid one; the LLM was weaker on the decision table, where it ignored the impossible combinations and the cap. The generated artefacts are good starting drafts but need the three checks above (reduce, mark impossible, add negative cases).

> *Prediction recorded for Task 9: pairwise testing will detect rule faults that involve two parameter values but not threshold faults on a single value.*

Task 6 – White-Box Testing and Test Adequacy: Statement, Branch and Mutation-Score Criteria

6.1 Coverage of the black-box suite measured with coverage.py

The suites built in Tasks 3 to 5 were run against Listing 1 with coverage.py (branch mode, version 7.17.1). Listing 1 has 19 executable statements (line 8, else:, is not a statement) and 16 branch outcomes (8 decisions × 2).

Suite	Tests	Statement coverage	Branch coverage	Mutants killed (score)
ECP robust suite (Task 3B)	11	89.5%	87.5%	43 / 69 = 62.3%
Boundary two-value suite (Task 4A)	21	84.2%	81.2%	50 / 69 = 72.5%
Decision-table suite (Task 5A)	12	94.7%	93.8%	48 / 69 = 69.6%
Pairwise suite (Task 5C)	11	89.5%	87.5%	41 / 69 = 59.4%
All four combined (54 distinct tests)	54	100.0%	100.0%	60 / 69 = 87.0%
Four-test statement-coverage suite (built for 6.3)	4	100.0%	100.0%	22 / 69 = 31.9%

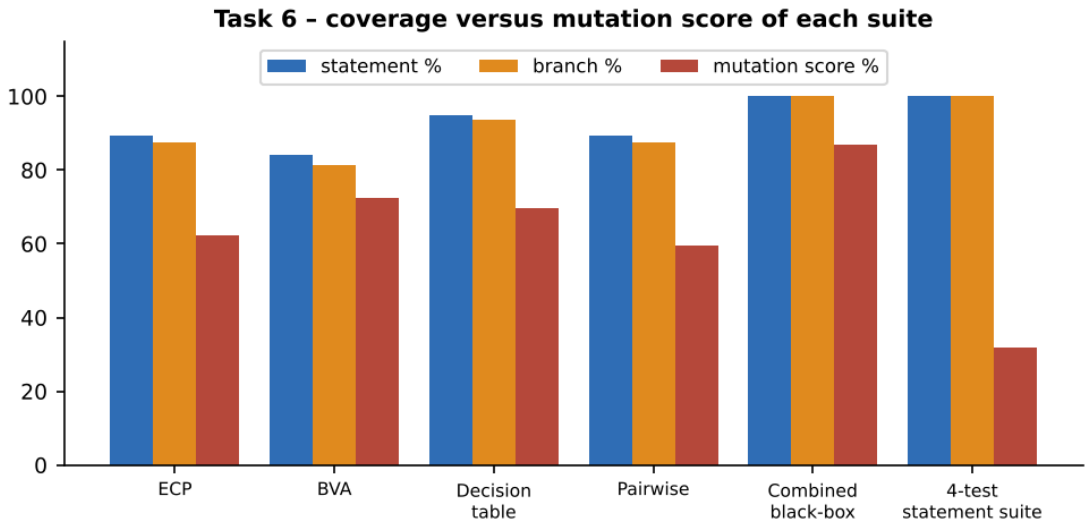


Figure 6.1 – Coverage versus mutation score

Uncovered statements and branches

Suite	Uncovered statements (line)	Branch outcomes not taken (decision line → target)
ECP robust suite	15, 17	line 14 → 15, line 16 → 17
Boundary two-value suite	15, 17, 19	line 14 → 15, line 16 → 17, line 18 → 19
Decision-table suite	3	line 2 → 3
Pairwise suite	3, 17	line 2 → 3, line 16 → 17

The ECP and boundary suites never apply the promo discount (line 15) and never reach the cap (line 17); the boundary suite also never produces WALLET_LOW (line 19) because all its boundary tests use CASH. The decision-table suite reaches 94.7% but never uses an invalid distance (line 3). The combination of the four suites covers every statement and branch: 100% / 100% (the assignment target was ≥ 90% branch coverage).

6.2 Mutation analysis

69 mutants of Listing 1 were generated automatically by a small mutation tool (mutants.py) that applies one change at a time: relational operator replacement (<=, <, >=, >, ==, !=), logical operator swap (and/or), arithmetic operator replacement (+/−, ×/÷) and constant change by ±1. Line 3, 19 and 20 have no mutable

operators. The combined black-box suite killed 60 mutants (87.0%). Nine survived and were classified by comparing each survivor with the original on the exhaustive input grid of 4,303,728 combinations of Section 0.3.

Mutant	Line	Statement	Change	Classification
M8	4	if km <= 2:	<= -> <	equivalent (no output difference in 4.3 M inputs)
M13	6	elif km <= 10:	<= -> <	equivalent (no output difference in 4.3 M inputs)
M15	6	elif km <= 10:	10 -> 9	REAL – survives because no test is near the limit
M32	10	if wait > 5:	> -> >=	equivalent (no output difference in 4.3 M inputs)
M34	10	if wait > 5:	5 -> 4	equivalent (no output difference in 4.3 M inputs)
M58	14	if promo == "RIDE50" and fare >= 200:	200 -> 199	REAL – survives because no test is near the limit
M61	16	if fare > 800:	> -> >=	equivalent (no output difference in 4.3 M inputs)
M62	16	if fare > 800:	800 -> 801	equivalent (no output difference in 4.3 M inputs)
M63	16	if fare > 800:	800 -> 799	equivalent (no output difference in 4.3 M inputs)

Seven survivors are equivalent mutants: km < 2 and km < 10 (continuous slabs), wait >= 5 and wait 5 → 4 (the first minute charged adds 0, as shown in Section 0.3), and the cap changes fare > 800 → >= 800, 801 and 799 (the cap only sets fare to 800; changing the comparison leaves every output on the grid unchanged). Two survivors are real: M15 (10 → 9 in line 6: rides of 9.1–10 km are priced with the slab-3 formula) and M58 (200 → 199 in the promo threshold). Both would be killed by one extra white-box test each (km 9.5; fare 199) – these are the two top-up tests used in Task 9.

Measure	Value
Mutants generated	69
Killed by the combined black-box suite	60
Equivalent mutants (survived, cannot be killed)	7
Real survivors	2
Mutation score (all mutants)	60 / 69 = 87.0%
Mutation score excluding equivalents	60 / 62 = 96.8%
After two top-up tests (Task 9 hybrid)	62 / 62 = 100% of killable mutants

6.3 Relationship between the three measures

A four-test suite was built only to execute every statement: km 0 (line 3), km 1.5 (line 5), km 6 with cash (lines 7 and 20) and km 60 with wait 30, hour 2, RIDE50, WALLET and balance 0 (lines 9, 11, 13, 15, 17 and 19).

Test	Input	Output
1	0 km | w0 | h12 | – | 0 | CASH	INVALID
2	1.5 km | w0 | h12 | – | 0 | CASH	50
3	6 km | w0 | h12 | – | 0 | CASH	98
4	60 km | w30 | h2 | RIDE50 | 0 | WALLET	WALLET_LOW

Result measured with coverage.py: 100% statement coverage and 100% branch coverage, but only 22 of 69 mutants killed: a mutation score of 31.9%. Every comparison in the program has been executed in both directions by values far from its limit, so the mutants that move a limit (<= to <, 200 to 199) are not noticed. Coverage tells which code ran; it does not tell whether the results were checked against the correct limits. In the reverse direction, a suite can have a high mutation score and still lack coverage only if it is very small, so

mutation score is the stronger measure and statement coverage the weakest: statement ≤ branch ≤ (roughly) mutation in the strength of the guarantee.

6.4 What each criterion cannot detect

- **Missing functionality.** The once-per-passenger rule of RIDE50 and the range checks for wait, hour and mode are not in Listing 1. No coverage figure, even 100%, can reveal them: the combined suite reached 100% statement, branch and condition coverage while six invalid-input classes are still accepted (Task 3).
- **Dependence on the program text.** All three criteria are defined on the code of Listing 1. If the code is rewritten (for example, a lookup table replaces the if/elif chain) the statements, branches and mutants all change, while the specification-based tests remain valid. Mutation scores also depend on the chosen operators: a mutant such as 146 → 140 is not generated by "±1" and would need a separate operator.
- **Oracle quality.** Coverage and mutation measure execution and observable differences, but a test with no assertion executes code and kills no mutant only if it asserts the output; 100% coverage with weak assertions is possible.
- **Equivalent mutants** inflate the denominator: here 7 of 69.

6.5 Release criterion proposed for SwiftRide

Criterion	Strength	Cost for this project	Suitable as release gate?
A Statement coverage	weak (4 tests gave 100% with 31.9% mutation)	19 statements, measured in 0.1 s	No – only a minimum sanity check
B Branch coverage	medium; shows untouched decisions	16 branch outcomes, measured in 0.1 s	Yes, as the first gate
C Mutation score	strongest; detects weak assertions and wrong limits	69 mutants × 17 tests ≈ 1,170 test executions, 0.02 s measured (generate, compile and run all 69 mutants)	Yes, for the fare function

Proposed release criterion: (1) all black-box tests pass; (2) branch coverage ≥ 95% on every module and 100% on Fare and Promo; (3) mutation score ≥ 90% of non-equivalent mutants on the fare function, with every surviving mutant reviewed and either killed or documented as equivalent. The thresholds are justified by the measurements: the combined suite reached 100% branch coverage and 100% of killable mutants, so the thresholds are achievable; the 2-second-per-test limit is not a constraint since each test takes well under 0.01 ms and the whole mutation run (69 mutants × 17 tests) was measured at 0.02 s. On a larger module (for example, Booking, with a database) the mutation run would be restricted to the modules changed in the release.

6.6 Critique: "100% coverage means the code is fully tested"

- Counter-example 1: the four-test suite has 100% statement and 100% branch coverage and kills only 31.9% of mutants.
- Counter-example 2: the 54-test combined suite also has 100% coverage but two real mutants survive and six invalid inputs are accepted by the unmodified program.
- Counter-example 3 (Task 1): structure-based tests with 100% branch coverage of lines 10, 12, 14 and 18 missed five of six seeded faults.
- The statement is therefore false. Coverage is a necessary condition (an unexecuted line certainly is untested) and a useful way to find gaps, but it is not a sufficient condition for adequacy.

> *Prediction recorded for Task 9: the basis-path suite will reach 100% statement and branch coverage but find clearly fewer seeded faults and have a lower mutation score than the black-box suite.*

Task 7 – Coverage Criteria: Decision, Condition/Decision and MC/DC

7.1 Decisions and simple conditions in Listing 1

Decision	Line	Expression	Simple conditions
D2	2	km <= 0 or km > 60	2a: km <= 0; 2b: km > 60
D4	4	km <= 2	4: km <= 2
D6	6	km <= 10	6: km <= 10
D10	10	wait > 5	10: wait > 5
D12	12	hour >= 23 or hour < 5	12a: hour >= 23; 12b: hour < 5
D14	14	promo == "RIDE50" and fare >= 200	14a: promo == "RIDE50"; 14b: fare >= 200
D16	16	fare > 800	16: fare > 800
D18	18	mode == "WALLET" and wallet < fare	18a: mode == "WALLET"; 18b: wallet < fare

Totals: 8 decisions (16 decision outcomes) and 12 simple conditions (24 condition outcomes). Four decisions (lines 2, 12, 14, 18) are compound. McCabe V(G) counting each condition = 13 (radon reports 13); counting decisions only = 9.

7.2 Minimum number of tests for lines 14 and 18

Criterion	Line 14 (promo and fare)	Line 18 (mode and wallet)	Reasoning
A Decision coverage	2 tests: (T,T) → true; (F,·) → false	2 tests	one test per outcome
B Condition/decision	2 tests in theory: (T,T) and (F,F); but with Python's short-circuit and, 14b is not evaluated when 14a is false, so the second condition never takes the value "false" in that test → 3 tests	3 tests (same argument)	each simple condition must take both values and be evaluated
C MC/DC	3 tests (n + 1): (T,T) → T; (F,T) → F; (T,F) → F	3 tests	each condition needs an independence pair

7.3 Three suites for the whole of Listing 1, executed

Suite A – decision coverage

Every decision true and false once (7 tests).

#	Input	Output
1	0 km | w0 | h12 | – | 0 | CASH	INVALID
2	1.5 km | w0 | h12 | – | 0 | CASH	50
3	6 km | w0 | h12 | – | 0 | CASH	98
4	20 km | w10 | h2 | RIDE50 | 0 | CASH	270
5	20 km | w0 | h12 | – | 0 | CASH	246
6	60 km | w30 | h2 | RIDE50 | 0 | WALLET	WALLET_LOW
7	5 km | w0 | h12 | – | 500 | WALLET	86

Suite B – condition/decision coverage

Suite A plus tests that make every simple condition take both values (14 tests).

#	Input	Output
1	0 km | w0 | h12 | – | 0 | CASH	INVALID
2	1.5 km | w0 | h12 | – | 0 | CASH	50
3	6 km | w0 | h12 | – | 0 | CASH	98

#	Input	Output
4	20 km | w10 | h2 | RIDE50 | 0 | CASH	270
5	20 km | w0 | h12 | – | 0 | CASH	246
6	60 km | w30 | h2 | RIDE50 | 0 | WALLET	WALLET_LOW
7	5 km | w0 | h12 | – | 500 | WALLET	86
8	61 km | w0 | h12 | – | 0 | CASH	INVALID
9	5 km | w0 | h23 | – | 0 | CASH	107.5
10	5 km | w0 | h2 | – | 0 | CASH	107.5
11	20 km | w0 | h12 | RIDE50 | 0 | CASH	196
12	5 km | w0 | h12 | – | 0 | CASH	86
13	5 km | w0 | h12 | – | 10 | WALLET	WALLET_LOW
14	5 km | w0 | h12 | – | 0 | UPI	86

Suite C – MC/DC for lines 2, 12, 14, 18

Suite B plus the extra tests for the independence pairs (16 distinct tests).

#	Input	Output
1	0 km | w0 | h12 | – | 0 | CASH	INVALID
2	1.5 km | w0 | h12 | – | 0 | CASH	50
3	6 km | w0 | h12 | – | 0 | CASH	98
4	20 km | w10 | h2 | RIDE50 | 0 | CASH	270
5	20 km | w0 | h12 | – | 0 | CASH	246
6	60 km | w30 | h2 | RIDE50 | 0 | WALLET	WALLET_LOW
7	5 km | w0 | h12 | – | 500 | WALLET	86
8	61 km | w0 | h12 | – | 0 | CASH	INVALID
9	5 km | w0 | h23 | – | 0 | CASH	107.5
10	5 km | w0 | h2 | – | 0 | CASH	107.5
11	20 km | w0 | h12 | RIDE50 | 0 | CASH	196
12	5 km | w0 | h12 | – | 0 | CASH	86
13	5 km | w0 | h12 | – | 10 | WALLET	WALLET_LOW
14	5 km | w0 | h12 | – | 0 | UPI	86
15	5 km | w0 | h12 | RIDE50 | 0 | CASH	86
16	5 km | w0 | h12 | – | 10 | CASH	86

Suite	Statement	Branch	Condition coverage (outcomes)	MC/DC achieved
A Decision (7 tests)	100.0%	100.0%	21 / 24 = 87.5%	D2 no, D12 no, D14 no, D18 yes
B Condition/decision (14)	100.0%	100.0%	23 / 24 = 95.8%	D2 yes, D12 yes, D14 no, D18 yes
C MC/DC (16)	100.0%	100.0%	24 / 24 = 100.0%	D2 yes, D12 yes, D14 yes, D18 yes

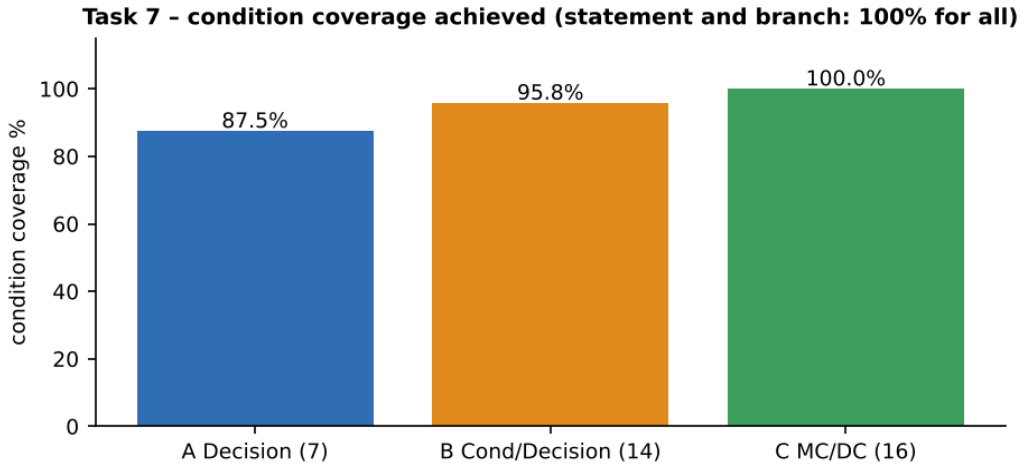


Figure 7.1 – Condition coverage of the three suites

Statement and branch coverage were measured with coverage.py; condition coverage and the MC/DC check were measured with an instrumented copy of Listing 1 that records every simple condition that is actually evaluated (short-circuit respected). All three suites already reach 100% statement and branch coverage: decision coverage is satisfied by suite A, and conditions are what separate the suites (87.5% → 95.8% → 100%).

7.4 MC/DC independence pairs

For each condition a pair of tests is shown whose only relevant difference is that condition and which gives different decision outcomes (unique-cause where possible; masking MC/DC where short-circuit makes the other condition unevaluated). Test numbers refer to suite C.

Condition	Test 1	Test 2	Type of pair
2a	C1: 0 km | w0 | h12 | – | 0 | CASH	C2: 1.5 km | w0 | h12 | – | 0 | CASH	masking
2b	C2: 1.5 km | w0 | h12 | – | 0 | CASH	C8: 61 km | w0 | h12 | – | 0 | CASH	unique-cause
12a	C2: 1.5 km | w0 | h12 | – | 0 | CASH	C9: 5 km | w0 | h23 | – | 0 | CASH	masking
12b	C2: 1.5 km | w0 | h12 | – | 0 | CASH	C4: 20 km | w10 | h2 | RIDE50 | 0 | CASH	unique-cause
14a	C2: 1.5 km | w0 | h12 | – | 0 | CASH	C4: 20 km | w10 | h2 | RIDE50 | 0 | CASH	masking
14b	C4: 20 km | w10 | h2 | RIDE50 | 0 | CASH	C15: 5 km | w0 | h12 | RIDE50 | 0 | CASH	unique-cause
18a	C2: 1.5 km | w0 | h12 | – | 0 | CASH	C6: 60 km | w30 | h2 | RIDE50 | 0 | WALLET	masking
18b	C6: 60 km | w30 | h2 | RIDE50 | 0 | WALLET	C7: 5 km | w0 | h12 | – | 500 | WALLET	unique-cause

Reading example, condition 12b (hour < 5): the two tests differ in hour (one has hour < 5, the other does not) while hour >= 23 is false in both, and line 12 changes outcome, so 12b alone decides the decision. The same reading applies to every row.

7.5 A decision-coverage suite that misses a fault which MC/DC detects

Fault M1: hour < 5 changed to hour < 0 in line 12, so early-morning rides (hours 0–4) lose the night surcharge. The decision-coverage suite {hour 23, hour 12} makes line 12 true once (through hour >= 23) and false once; the mutant gives the same outputs as the original on both tests, so **decision coverage does not detect it** (detected: no). Adding hour 2 – the test of the MC/DC pair for condition 12b – exposes the fault (detected: yes). MC/DC is not a boundary criterion: the fault hour < 5 → hour < 4 is still undetected by the three MC/DC tests (detected: no) and needs hour 4, a boundary value from Task 4 (detected once hour 4 is added: yes).

Condition-level faults detected by each suite

Fault	A Decision	B Cond/decision	C MC/DC
M1 hour<5 -> hour<0	found	found	found
M2 hour>=23 -> hour>=24	MISSED	found	found
M3 promo== -> promo!=	found	found	found

Fault	A Decision	B Cond/decision	C MC/DC
M4 fare>=200 -> fare>=0	MISSED	MISSED	found
M5 mode==WALLET -> True	found	found	found
M6 km>60 -> km>600	MISSED	found	found

Totals: A 3 of 6, B 5 of 6, C 6 of 6. Fault M4 (fare >= 200 → fare >= 0, the promo threshold removed) is found only by MC/DC, because it needs the independence pair of line 14 (promo true with fare below 200).

7.6 Effect of short-circuit evaluation

Line	Short-circuit effect	Combination that cannot be exercised
2	if km <= 0 is true, km > 60 is not evaluated	(T,T) is also logically impossible (km ≤ 0 and km > 60)
12	if hour >= 23 is true, hour < 5 is not evaluated	(T,T) impossible (hour ≥ 23 and hour < 5); with 12b evaluated only when 12a is false
14	if promo is not RIDE50, the fare is not looked at	14b can be observed only in tests that already have the promo code, so a “promo absent, fare ≥ 200” test shows nothing about 14b (needed for the pair)
18	if mode is not WALLET, the wallet is not compared	18b is observed only for WALLET rides; the test “UPI with low balance” cannot show 18b true

The consequence is that “condition coverage” of the second condition of an AND (or OR) is meaningful only with tests that make the first condition true (or false); the number of tests rises from 2 to 3. MC/DC with short-circuit evaluation is naturally the masking form.

7.7 Cost–benefit and recommendation

Criterion	Tests (Listing 1)	Condition faults found (of 6)	Cost	Use in SwiftRide
A Decision	7	3	lowest	less critical code (UI formatting, reminders)
B Condition/decision	14	5	medium	Booking and Ride Management rules
C MC/DC	16 (n + 1 per decision)	6	a few extra tests: for 4 compound decisions it costs 9 tests more than A	fare function, promo and wallet checks

MC/DC adds only two tests over B for this function and finds the two faults that are expensive in money (promo threshold, night surcharge), so it is applied to calculate_fare and the payment check. It is not worth full MC/DC on the less critical code, where decision coverage is enough.

7.8 Reliability of an LLM-generated MC/DC table for line 12

Prompt: “Give an MC/DC test table for hour >= 23 or hour < 5.” One-pass answer: three tests – hour 23 (A = T, B = F → T), hour 2 (A = F, B = T → T), hour 12 (A = F, B = F → F) – with independence pairs (hour 23, hour 12) for A and (hour 2, hour 12) for B. Each pair was checked by running the instrumented function:

Pair claimed	Condition	Condition values	Decision outcomes	Other condition fixed?	Verdict
(23, 12)	12a: hour >= 23	T → F	T → F	12b: F in both	independent ✓
(2, 12)	12b: hour < 5	T → F	T → F	12a: F in both	independent ✓

In this run the table is correct: the (T,T) row that would be impossible was not used and all pairs are valid. The risk remains for AND/OR conditions with short-circuit evaluation (line 14, 18), where an LLM table that evaluates all conditions will list pairs in which the second condition is never evaluated; each pair therefore has to be checked mechanically as done here.

> *Prediction recorded for Task 9: MC/DC adds boundary-insensitive tests; it will not replace boundary tests.*

Task 8 – Control Flow Graphs: Statement-Level, Basic-Block and DD-Path Graphs of find_driver

8.1 Listing 2 with node labels

```
1  def find_driver(ids, target):
2      low = 0
3      high = len(ids) - 1
4      while low <= high:            # predicate 1
5          mid = (low + high) // 2
6          if ids[mid] == target:    # predicate 2
7              return mid
8          elif ids[mid] < target:   # predicate 3
9              low = mid + 1
10         else:
11             high = mid - 1
12     return -1
```

Nodes are labelled with the line numbers of the listing. Line 1 (the def) is represented by the Entry node; line 10 (else:) is not a statement and gets no node.

8.2 The three graphs

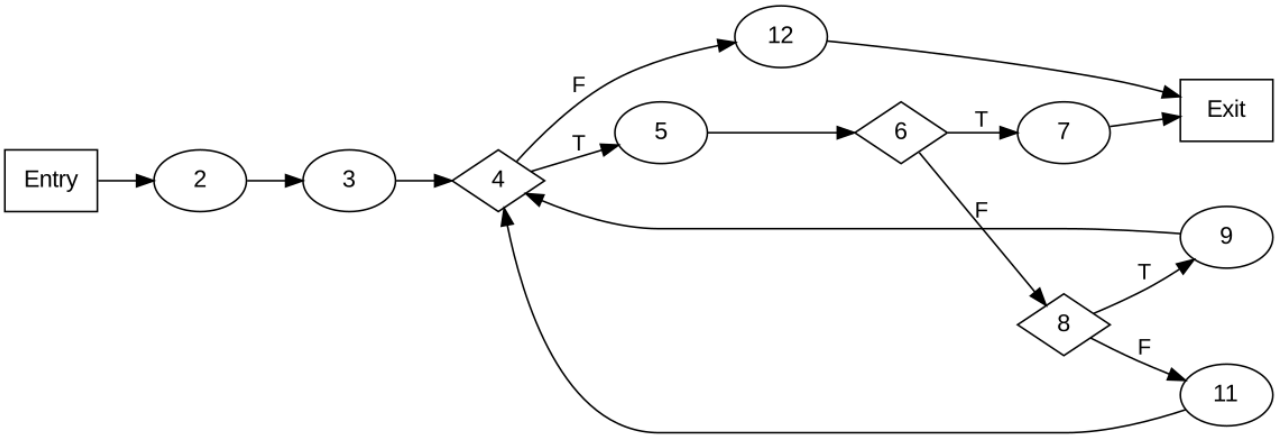


Figure 8.1 – (A) Statement-level control flow graph of find_driver (12 nodes, 14 edges; diamonds are predicates, T/F label their outcomes)

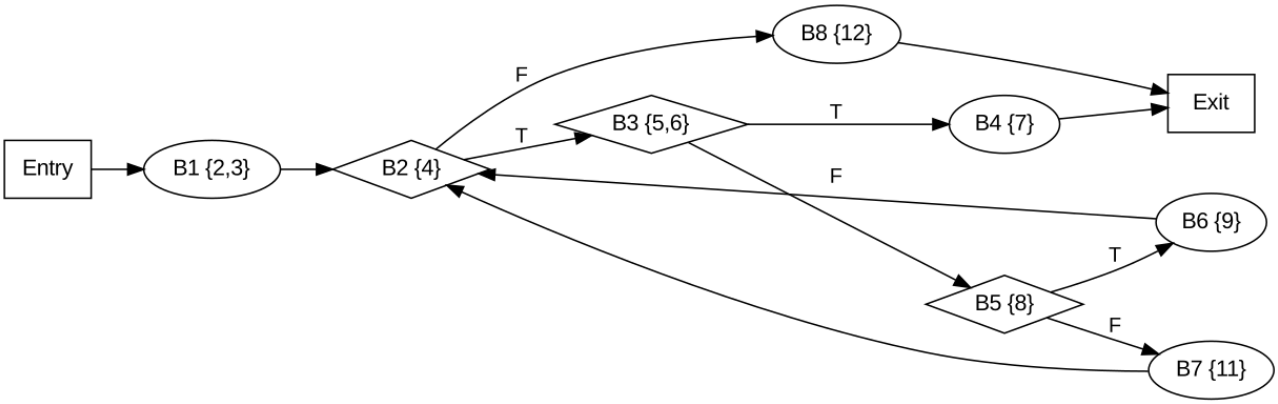


Figure 8.2 – (B) Basic-block graph of find_driver (10 nodes, 12 edges; block contents in braces are line numbers)

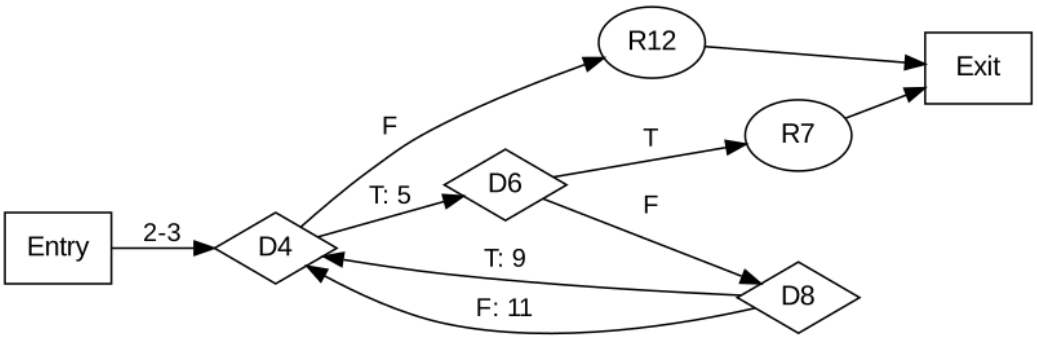


Figure 8.3 – (C) DD-path graph of find_driver (7 nodes, 9 edges; the two edges from D8 back to D4 are the paths through line 9 and line 11)

Graph	Nodes (N)	Meaning of a node	Edges (E)	Edge meaning
A Statement-level	12: Entry, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, Exit	one executable statement	14	control passing from one statement to the next
B Basic-block	10: Entry, B1{2,3}, B2{4}, B3{5,6}, B4{7}, B5{8}, B6{9}, B7{11}, B8{12}, Exit	maximal straight-line sequence; a decision ends a block	12	transfer between blocks
C DD-path	7: Entry, D4, D6, D8, R7 (return mid), R12 (return −1), Exit	decision, entry, exit or return	9	the straight path (lines in between) from one decision to the next

8.3 Cyclomatic complexity V(G) by three methods

Graph	E − N + 2	Predicate nodes + 1	Regions (bounded + outer)	V(G)
A Statement-level	14 − 12 + 2 = 4	3 + 1 = 4	3 inside + 1 outside = 4	4
B Basic-block	12 − 10 + 2 = 4	3 + 1 = 4	4	4
C DD-path	9 − 7 + 2 = 4	3 + 1 = 4	4	4

Predicates: the while condition (line 4) and the two if conditions (lines 6 and 8). Regions: the loop forms region 1 (while → if → elif → back to while through line 9), region 2 (through line 11) and region 3 (the found-return branch, closed by the exits), and the unbounded outer region is the fourth. The tool radon independently reports complexity 4 for find_driver (and 13 for calculate_fare, where each compound condition counts once per condition; 9 if only whole decisions are counted).

8.4 Effect of granularity

Property	A Statement-level	B Basic-block	C DD-path
Nodes / edges	12 / 14	10 / 12	7 / 9
V(G)	4	4	4
Ease of spotting paths	long sequences to follow	shorter; straight code merged	easiest; each path is a sequence of decisions
Closeness to the code	1 node per line; easy to map to coverage.py line data	blocks map to coverage arcs	needs a short explanation of edge labels
Growth with program size	linear in lines	linear in blocks	linear in decisions

The size of the graph falls from 12 to 7 nodes as granularity gets coarser, and paths are easier to identify, but V(G) does not change: merging nodes that have exactly one successor removes one node and one edge each time, so E − N is unchanged. V(G) therefore depends on the decision structure, not on the drawing.

8.5 Independent (basis) paths and test data

Path	Lines executed (traced by Python)	Input (sorted list, target)	Result	Description
P1	2 → 3 → 4 → 12	ids = [], target = 5	-1	empty list, loop never entered
P2	2 → 3 → 4 → 5 → 6 → 7	ids = [7], target = 7	0	target at first midpoint

Path	Lines executed (traced by Python)	Input (sorted list, target)	Result	Description
P3	2 → 3 → 4 → 5 → 6 → 8 → 9 → 4 → 5 → 6 → 7	ids = [1, 3, 5], target = 5	2	go right (elif), then found
P4	2 → 3 → 4 → 5 → 6 → 8 → 11 → 4 → 5 → 6 → 7	ids = [1, 3, 5], target = 1	0	go left (else), then found

The four paths are linearly independent (each uses at least one edge the earlier ones do not use: P1 the while-false exit, P2 the found branch, P3 the elif branch with back-edge via line 9, P4 the else branch with back-edge via line 11) and number exactly V(G) = 4. All four are feasible. Further paths, for example the “not found after the loop” path:

Extra path	Input	Lines executed	Result
X1	ids = [1, 3, 5, 7], target = 4	2 → 3 → 4 → 5 → 6 → 8 → 9 → 4 → 5 → 6 → 8 → 11 → 4 → 12	-1
X2	ids = [1], target = 2	2 → 3 → 4 → 5 → 6 → 8 → 9 → 4 → 12	-1
X3	ids = [5], target = 1	2 → 3 → 4 → 5 → 6 → 8 → 11 → 4 → 12	-1

are combinations of the basis paths. Because of the loop, the number of distinct paths is not finite in principle; with a sorted list of n IDs the loop runs at most ⌊log2 n⌋ + 1 times, so for the 4,000 drivers of SwiftRide at most 12 iterations. A path that cannot be executed with a sorted list is any path that moves right and later finds an element smaller than an earlier midpoint (needs an unsorted list): such paths are infeasible under the precondition “ids is sorted” and are not basis paths. Correctness of the function was also confirmed by an exhaustive check over every sorted list of up to 6 elements from 0–7 and every target from −1 to 8 (all results correct).

8.6 Recommendation: standard granularity for the team

Criterion	A Statement	B Basic-block	C DD-path
Accuracy (V(G))	correct	correct	correct
Readability	poor for long functions	good	best – one node per decision
Tool support	coverage.py reports statements; most CFG tools draw this level	coverage.py arcs and Python “dis” blocks map directly	few tools; can be produced from block graphs

Chosen standard: the **DD-path graph for design and review** (path selection, basis paths, V(G), documentation – the form used for Listing 1 in Task 9) and the **basic-block graph as the tool-generated check**. The statement-level graph is used only for teaching and for very short functions.

8.7 Evaluating a control flow graph produced by an LLM

Prompt: “Draw the control flow graph of find_driver and compute V(G).” The one-pass answer was a statement-level graph with 12 nodes (Start, lines 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, End) and 14 edges, V(G) = 14 − 12 + 2 = 4. Each element was checked against the traced paths above:

Check	Result
Missing edges	none – both loop-back edges (9 → 4 and 11 → 4) and the 4 → 12 exit are present
Extra nodes	none (line 10, else, correctly has no node)
V(G)	4 – correct, equal to the radon value
Edge labels (T/F)	present for lines 4, 6 and 8

This one-pass LLM output had no error. The common failure points of LLM graphs – a missing back edge, a node for else, or a graph that sends line 7 to line 12 instead of Exit – did not occur here but each has to be checked, because a missing back edge would give V(G) = 3 and one basis path too few (the else/elif path with a second iteration).

Task 9 – Path Testing and the Integrated Test Design

9.1 Ten seeded faults

ID	Line	Fault	Kind
X1	2	km <= 0 → km < 0	boundary (lower limit)
X2	4	km <= 2 → km <= 2.1	shifted slab limit
X3	6	km <= 10 → km <= 10.1	shifted slab limit
X4	10	wait > 5 → wait > 6	shifted free-wait limit
X5	12	hour >= 23 → hour > 23	night start
X6	13	night factor 1.25 → 1.2	wrong constant
X7	14	promo threshold 200 → 150	wrong constant
X8	16–17	cap 800 → 900	wrong constant
X9	18	wallet < fare → wallet <= fare	wrong operator
X10	12–15	promo applied before the night factor	interaction / order

9.2 Control flow graph of Listing 1 and basis paths

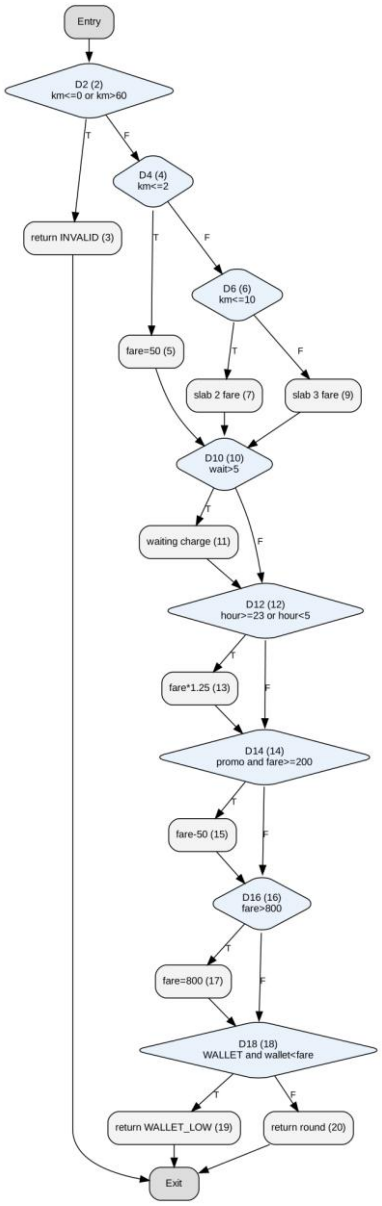


Figure 9.1 – Decision-level control flow graph of Listing 1 (line numbers in brackets)

Listing 1 has 8 decisions, so V(G) = 8 + 1 = 9 (decision level). If every simple condition is counted, V(G) = 13 (radon). Nine independent paths are needed; nine tests were derived, each flipping one decision of the baseline path P1 (slab 3, no wait, day, promo applied, no cap, no wallet check):

Path	Test data	Purpose	Output	Outcomes D2, D4, D6, D10, D12, D14, D16, D18
P1	20 km | w0 | h12 | RIDE50 | 0 | CASH	baseline: slab 3, no wait, day, promo applied, no cap, no wallet check	196	F, F, F, F, F, T, F, F
P2	0 km | w0 | h12 | – | 0 | CASH	flip D2: km<=0 or km>60 -> INVALID	INVALID	T, –, –, –, –, –, –, –
P3	1.5 km | w0 | h12 | RIDE50 | 0 | CASH	flip D4: km<=2 (promo cannot apply)	50	F, T, –, F, F, F, F, F
P4	6 km | w0 | h12 | RIDE50 | 0 | CASH	flip D6: 2<km<=10 (promo cannot apply)	98	F, F, T, F, F, F, F, F
P5	20 km | w10 | h12 | RIDE50 | 0 | CASH	flip D10: waiting charge	206	F, F, F, T, F, T, F, F
P6	20 km | w0 | h23 | RIDE50 | 0 | CASH	flip D12: night surcharge	257.5	F, F, F, F, T, T, F, F
P7	20 km | w0 | h12 | – | 0 | CASH	flip D14: no promo	246	F, F, F, F, F, F, F, F
P8	60 km | w30 | h2 | RIDE50 | 0 | CASH	flip D16: cap applied	800	F, F, F, T, T, T, T, F
P9	20 km | w0 | h12 | RIDE50 | 10 | WALLET	flip D18: WALLET_LOW	WALLET_LOW	F, F, F, F, F, T, F, T

The nine path vectors (outcome of every decision as coordinates) have rank 9, which proves they are linearly independent and form a basis. P3 and P4 are not pure single flips of P1: when the slab changes to km ≤ 2 or km ≤ 10, D14 must also take the false outcome, because the promo cannot be applied there (see the next section).

9.3 Infeasible paths

Quantity	Value
Structural paths of Listing 1 (1 invalid path + 3 slabs × 2^5 outcomes of D10, D12, D14, D16, D18)	97
Feasible paths found (inputs tried: 133 distances × 5 waits × 6 hours × 2 promo × 4 wallet/mode)	41
Infeasible paths	56
promo applied (D14 = T) with km ≤ 2 (D4 = T)	16
cap applied (D16 = T) with km ≤ 2 (D4 = T), promo not applied	8
cap applied (D16 = T) with 2 < km ≤ 10 (D6 = T)	16
other: promo or cap needing wait/night that the path excludes	16

Infeasible path	Why it cannot be executed	Calculation
Promo applied on a ride of up to 2 km (D4 = T, D14 = T)	the fare can never reach ₹200	maximum fare for km ≤ 2: (50 + (30−5)×2) × 1.25 = 100 × 1.25 = ₹125 < 200
Cap applied on a ride of up to 10 km (D6 = T, D16 = T)	the fare can never exceed ₹800	maximum fare for km ≤ 10: (146 + 50) × 1.25 = ₹245 < 800
Cap on slab 3 with D10 = F and D12 = F	without waiting and night the fare of a ride of up to 60 km is at most 146 + 500 = ₹646	646 < 800
Promo applied on 2–10 km with no wait and no night (D6 = T, D10 = F, D12 = F, D14 = T)	maximum 146 < 200	146 < 200

The infeasible-path list was found by trying a dense grid of inputs and then confirmed analytically by the maximum-fare calculations above; the grid is not an exhaustive proof, the arithmetic is. Infeasible paths matter because path coverage of 97 structural paths would be impossible; the basis set of 9 contains only feasible paths and the feasible total is 41.

9.4 The three suites, plus reference suites, measured

Suite A = the nine basis-path tests. Suite B = all black-box tests of Tasks 3–5 (ECP robust 11, two-value BVA 21, decision table 12, pairwise 11; 54 distinct tests). Suite C = B plus two white-box tests added only for what B left behind: the survivors of the mutation run (km 9.5 kills 10 → 9 at line 6; fare 199 – km 14.5, wait 9 – kills the 200 → 199 survivor); B already had 100% statement, branch and condition coverage, so no coverage-driven test was needed. C′ is C reduced by a greedy set-cover to the smallest subset that keeps every covered decision outcome and every killed mutant. D is a model-based suite: pairwise combinations of boundary-rich values, expected values from the oracle. E is the LLM one-pass suite of Task 1. R is 54 uniform random tests. Mutation score is over the 69 mutants of Task 6.

Suite	Tests	Stmt	Branch	Cond.	Mutation score	Faults found	Run time (in-process)
A Basis-path	9	100%	100%	92%	69.6%	5 / 10	0.003 ms
B Black-box + pairwise	54	100%	100%	100%	87.0%	10 / 10	0.018 ms
C Hybrid	56	100%	100%	100%	89.9%	10 / 10	0.018 ms
C' Hybrid minimised	17	100%	100%	100%	89.9%	10 / 10	0.006 ms
D Model-based (pairwise, oracle)	48	100%	100%	100%	82.6%	10 / 10	0.018 ms
E LLM-style (unreviewed)	17	100%	100%	100%	71.0%	4 / 10	0.004 ms
R Random (54 tests)	54	95%	94%	92%	68.1%	5 / 10	0.033 ms

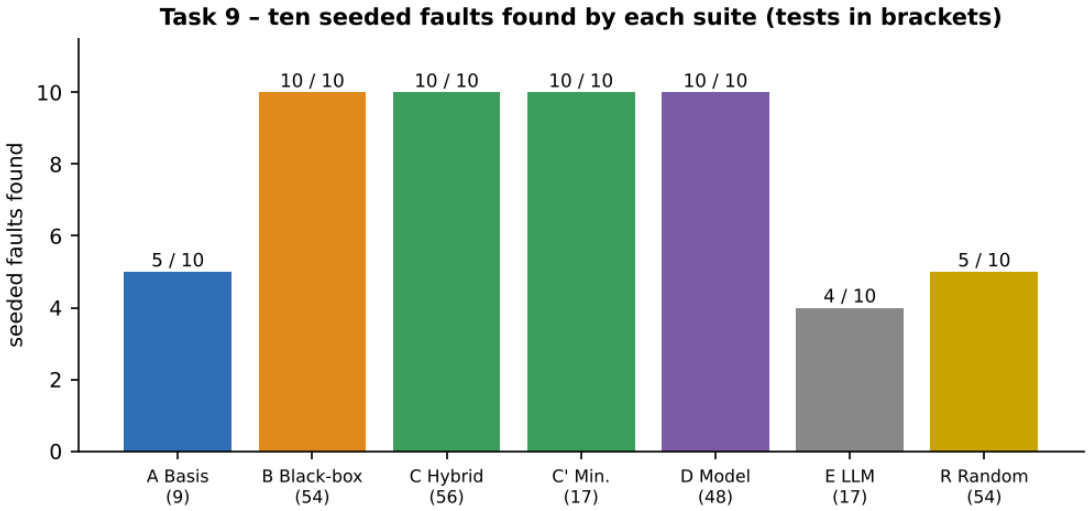


Figure 9.2 – Seeded faults found by each suite

Fault	A Basis	B Black-box	C Hybrid	C′ Min.	D Model	E LLM	R Random
X1	✓	✓	✓	✓	✓	✓	✗
X2	✗	✓	✓	✓	✓	✗	✗
X3	✗	✓	✓	✓	✓	✗	✗
X4	✗	✓	✓	✓	✓	✗	✗
X5	✓	✓	✓	✓	✓	✓	✓
X6	✓	✓	✓	✓	✓	✓	✓
X7	✗	✓	✓	✓	✓	✗	✓
X8	✓	✓	✓	✓	✓	✓	✓
X9	✗	✓	✓	✓	✓	✗	✗
X10	✓	✓	✓	✓	✓	✗	✓

Observations: the basis-path suite covers 100% of statements and branches with only nine tests and kills 69.6% of mutants, but finds only 5 of 10 faults – the five it misses (X2, X3, X4, X7, X9) are exactly the faults that need a value at a limit (2.1, 10.1, 6 minutes, fare 150–199, balance equal to the fare). B and C find all ten.

C′ (17 tests) keeps every fault and every mutant kill of the 56-test suite C, which means about two thirds of the black-box suite is redundant for this function. The LLM suite reaches 100% coverage with 17 tests and finds 4 of 10; random testing with the same number of tests as B reaches 94% branch coverage and finds 5 of 10.

9.5 Effort and maintainability when a rule changes (night factor 1.3, starting at 10 pm)

Effort for the three suites (author's estimates, minutes of tester time, including checking expected values): A 54 (9 tests × 6), B 240 (54 tests, partly generated), C 255, C′ 255 + 30 for the reduction. When the rule changes, two things happen to a suite: expected values change, and the boundary tests (which sit at the old limit 22/23) become wrong. Both were counted by recomputing every expected value with the new rule:

Suite	Tests	Expected values that change	Share	Tests at hour 22/23 (boundary moved to 21/22)
A Basis-path	9	1	11%	1
B Black-box + pairwise	54	17	31%	9
C Hybrid	56	17	30%	9
C' Hybrid minimised	17	4	24%	3
D Model-based (pairwise, oracle)	48	21	44%	16
E LLM-style (unreviewed)	17	2	12%	2
R Random (54 tests)	54	16	30%	7

The basis-path suite is the cheapest to maintain (1 of 9 expected values changes) because it uses a few mid-range values, but that is also why it detects less. The black-box suite B needs 17 expected values corrected and 9 boundary tests moved. The minimised hybrid C′ needs 4 corrections and 3 boundary moves, which is the lowest rework per fault found. The model-based suite D changes in 21 expectations but all of them are regenerated automatically from the oracle after one edit of the model, which is its main maintenance advantage.

9.6 Measured results against the predictions of Tasks 1–8

Prediction (Task)	What was measured in Task 9	Verdict
1: a code-only suite finds clearly fewer faults than specification + fault-based design	A basis suite 5/10, E LLM suite 4/10; B, C, D 10/10	supported
2: 54 uniform random tests find about half of the faults and no boundary fault reliably	R found 5/10 (it hit X5, X6, X7, X8 and X10 by chance, none of X1–X4 limit faults); 94% branch coverage	supported
3: ECP alone misses the promo-before-night interaction	ECP suite found 3/10; X10 missed	supported
4: two-value BVA finds all boundary-type faults	BVA found 6/10 including all of X1–X5; it missed X7–X10 (promo, cap, wallet, order), which are not boundary-type faults of km/wait/hour	supported (and BVA alone is insufficient)
5: pairwise finds two-value faults but not single-value threshold faults	pairwise found 3/10: X5, X6, X10; it missed X1–X4 and X7–X9	supported; it also found X10, which ECP missed
6: basis-path suite has 100% coverage but finds fewer faults	A: 100% statement and branch, 5/10 faults, 69.6% mutation	supported
7: MC/DC does not replace boundary tests	hour 4 needed for hour < 4 (Task 7); X5 found only by tests at hour 23	supported

9.7 Test-design workflow and Design Decision Table

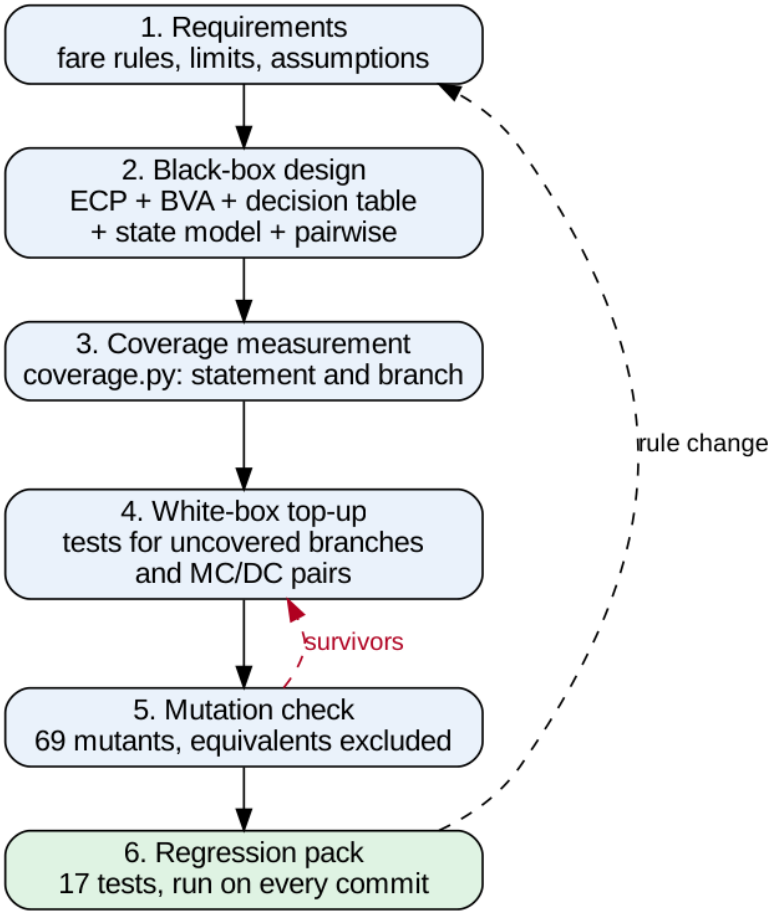


Figure 9.3 – Test-design workflow used for SwiftRide (original diagram)

Decision	Justification (evidence)	What if not
Start from the specification and design black-box tests first	A (spec) and C (fault-based) found 6/6 and B (structure) 1/6 in Task 1; basis paths found 5/10 in Task 9	Missing-functionality faults (once-only promo, input validation) are never tested
Use two-value boundary tests, not robust or worst-case	6/6 faults with 21 tests versus 3/6 (19) and 2/6 (125) in Task 4	Limit faults survive; the suite is large but blind to slab limits
Use robust equivalence classes, not valid-only	6 of 8 invalid classes are accepted by Listing 1 (Task 3)	Validation defects stay undiscovered
Use a decision table for promo/cap/payment and a state model for the ride	Decision table found both rule faults, pairwise missed RL2; state tests found all 3 life-cycle faults (Task 5)	Rule-combination and sequence defects (complaints 3, 4, 5) remain
Measure statement, branch and condition coverage after the black-box design	Combined black-box suite reached 100% / 100% / 100% with no extra test	Coverage gaps go unseen; but coverage alone is not used as the criterion
Run mutation analysis as the adequacy check	100% coverage but 31.9% mutation for the 4-test suite; 2 real survivors found in B	A “100% covered” suite with weak limits would pass the release gate
Add white-box tests only for uncovered conditions and surviving mutants	2 added tests killed the 2 real survivors (Task 6, hybrid C)	Either redundant tests inflate the suite, or real survivors stay
Minimise the suite for the regression pack (C′, 17 tests)	Same 10/10 faults and 62 mutants killed as the 56-test suite; 4 expected values change on the rule change	Regression pack of 56 tests costs more to maintain with no gain in detection
Use MC/DC on calculate_fare only	MC/DC found 6/6 condition faults against 3/6 for decision coverage; costs 9 more tests (Task 7)	Condition-level faults (hour < 5 stuck, threshold removed) escape; applying it everywhere wastes effort
Keep the oracle independent of the code	Oracle written from the rules found 96/3600 raw float mismatches that would have failed naive equality (Task 4)	Tests pass faulty code, or fail correct code because of rounding

9.8 Path testing applied to two other programs

Property	calculate_fare (SwiftRide)	Train-ticket concession (train.py)	Date validation (datev.py)	find_driver (Listing 2)
Decision structure	8 sequential decisions after the validity check; 4 compound	8 decisions, mostly if/elif chain on age, then two independent surcharges; 4 compound	7 decisions: range checks, month group (4 conditions), February with leap-year rule (3 conditions)	loop with 3 predicates
V(G), every condition counted (radon)	13	14	14	4
V(G), decisions only	9	9	7	4
Basis-path tests (V(G), condition level)	13 (9 at decision level)	14	14	4
Infeasible paths	yes – 56 of 97 structural paths (fare ceilings)	some: a child under 5 pays 0, so “fare > 0” can never be true after that branch (disabled quota or AC surcharge on a free ticket is infeasible)	some: in the leap-year condition, year % 400 = 0 together with year % 4 ≠ 0 is impossible	none for sorted input
Tests designed in this report (BVA + ECP)	54 (black-box) / 17 minimised	22 tests: 100% statement, 100% branch	25 tests: 100% statement, 100% branch	4 basis + 3 extra

Changes in structure: the train-ticket concession is a **priority chain** (age bands tested in order); V(G) comes from the number of branches of the chain and independent surcharges, and most paths are feasible, so the basis set is close to the full set. The date routine has the **leap-year rule as a compound condition nested in a branch** (V(G) = 14 for 7 decisions); here MC/DC and the century boundaries (1900, 2000) matter more than path count. SwiftRide’s fare function has the largest share of infeasible paths because its decisions are linked by arithmetic (a ceiling on the fare). Test counts: basis testing needs V(G) tests (13, 14, 14, 4); a black-box design for the train program needed 22 and for the date routine 25 tests, which both reach 100% statement and branch coverage in this execution.

9.9 Fourth option: model-based and LLM-assisted suites, and the final suite

Criterion	A Basis-path	B Black-box + pairwise	C′ Hybrid (minimised)	D Model-based	E LLM-assisted (unreviewed)
Faults found of 10	5	10	10	10	4
Tests	9	54	17	48	17
Mutation score	69.6%	87.0%	89.9%	82.6%	71.0%
Effort to create (estimate)	54 min	240 min	285 min	120 min (model) + generator	10 min + 60 min review
Traceability to requirements	low (paths, not rules)	high (each test names its class/rule)	high	highest (tests generated from the rule model)	none unless added by hand
Maintenance on a rule change	1 test changes, but misses faults	17 expectations and 9 boundary tests	4 expectations, 3 boundary tests	edit the model once, regenerate	must be regenerated and re-reviewed; no memory of the previous reasoning

Final suite (defended): the pre-release suite is **B plus the two white-box top-ups (suite C, 56 tests)**, because it has the highest measured mutation score (89.9%), finds 10 of 10 faults and 100% statement, branch and condition coverage; the **daily regression pack is the 17-test minimised C′**, which has the same detection and mutation kills at one third of the size and the lowest rework on a rule change. Model-based generation (D) is kept as the method for the new rules of Task 10, because it is the only option whose cost does not grow with the number of rules. Basis-path testing is used to measure complexity and find infeasible paths, not as the test design. The LLM suite is accepted only after review.

Task 10 – Re-apply and Adapt the Test Design to the Changed Requirement

10.1 Develop: the extended prototype

The prototype of Listing 1 was extended with the new rules (calculate_fare2 in fare2_src.py). The assignment does not fix the amounts and order of the new rules, so the following assumptions are stated and justified; they are implemented in the code and in an independent Decimal-based oracle (oracle2) written separately from the same list.

Rule	Assumption used	Why
Shared ride	pooled = number of co-passengers, allowed values 0, 2, 3; fare × 0.85 for 2 co-passengers, × 0.75 for 3	a discount that grows with sharing; values are not given in the assignment
Surge	multiplier 1.0 – 3.0 in steps of 0.1 (other values INVALID); applied to base fare + waiting, before the night factor	surge is a demand effect on the ride itself, not on tolls or taxes
Order of operations	slab → waiting → surge → night → pool discount → promo → loyalty → cap ₹800 → toll → airport ₹100 → GST 5% → wallet check	keeps the Task 9 core in the same order; toll, airport and GST are outside the cap and are not surged or discounted
Promo	RIDE50 as before (−₹50 if fare ≥ 200); new MORNING20: 20% off between 7:00 and 9:59 if fare ≥ ₹150	a time-window promo as in the requirement
Loyalty tier	NONE, SILVER 3%, GOLD 6%, PLATINUM 10% applied after the promo	discount on what the passenger actually pays for the ride
Toll / airport	toll ≥ 0 added as given; airport adds ₹100	flat charges
GST	5% on the total including toll and airport	tax on the final amount
Validation	new INVALID cases for wait, hour, passengers, pooled, surge, toll, mode and tier	the robust partitioning of Task 3 showed these checks are missing in Listing 1

The extended function has V(G) = **37** (radon, every condition counted; the assignment estimates about 35 decision points). The commented code:

```
def calculate_fare2(km, wait, hour, promo, wallet, mode, pooled=0, surge=1.0, toll=0, airport=False,
tier="NONE", pax=1):
    # --- validation (new: wait, hour, passengers, pooled, surge step, toll, mode, tier)
    if km <= 0 or km > 60:
        return "INVALID"
    if wait < 0 or wait > 30:
        return "INVALID"
    if hour < 0 or hour > 23:
        return "INVALID"
    if pax < 1 or pax > 4:
        return "INVALID"
    if pooled not in (0, 2, 3):
        return "INVALID"
    if surge < 1.0 or surge > 3.0 or abs(surge * 10 - round(surge * 10)) > 1e-9:
        return "INVALID"
    if toll < 0:
        return "INVALID"
    if mode not in ("WALLET", "UPI", "CASH"):
        return "INVALID"
    if tier not in ("NONE", "SILVER", "GOLD", "PLATINUM"):
        return "INVALID"
    # --- base fare by distance slab (unchanged from Listing 1)
    if km <= 2:
        fare = 50
    elif km <= 10:
        fare = 50 + (km - 2) * 12
    else:
        fare = 146 + (km - 10) * 10
    if wait > 5:
        fare = fare + (wait - 5) * 2
    # --- surge multiplier 1.0..3.0 (new)
    if surge > 1.0:
        fare = fare * surge
    if hour >= 23 or hour < 5:
        fare = fare * 1.25
    # --- shared-ride discount (new): 2 co-passengers 15%, 3 co-passengers 25%
    if pooled == 2:
```

```
        fare = fare * 0.85
    elif pooled == 3:
        fare = fare * 0.75
    # --- promos: RIDE50 as before; MORNING20 time-window promo (new)
    if promo == "RIDE50" and fare >= 200:
        fare = fare - 50
    elif promo == "MORNING20" and 7 <= hour < 10 and fare >= 150:
        fare = fare * 0.8
    # --- loyalty tier discount (new)
    if tier == "SILVER":
        fare = fare * 0.97
    elif tier == "GOLD":
        fare = fare * 0.94
    elif tier == "PLATINUM":
        fare = fare * 0.90
    # --- cap applies before toll, airport and GST
    if fare > 800:
        fare = 800
    # --- toll and airport charges (new), then 5% GST (new)
    if toll > 0:
        fare = fare + toll
    if airport:
        fare = fare + 100
    fare = fare * 1.05
    # --- wallet check against the final payable amount
    if mode == "WALLET" and wallet < fare:
        return "WALLET_LOW"
    return round(fare, 2)
```

10.2 Impact analysis – which Task 9 assumptions no longer hold

Task 9 assumption	Value then	Value now	Consequence
Number of paths is small	V(G) 13 (9 at decision level); 97 structural paths, 41 feasible	V(G) = 37; about 13,824 structural paths (3 slabs × 2 × 2 × 2 × 3 pool × 3 promo × 4 tiers × 2 × 2 × 2 × 2)	basis testing needs 37 tests; path coverage is impossible within the time limit (13,824 × 0.5 s = 1.9 hours)
Inputs are independent	promo and night were the only interaction	surge × night × pool × promo × tier × cap interact; order of operations matters	pairwise/orthogonal coverage of the new parameters is needed; N2, N10, N11, N12 are order faults
Run time is negligible	56 tests ≈ 28 s at 0.5 s/test	10-minute budget = 1,200 tests at 0.5 s/test	every added technique must be weighed in seconds
Specification is stable	one rule change was assumed (night factor)	weekly releases; GST changes every expected value	expected values must be generated from a model, not typed in
Fare is monotone in distance	true for no-promo rides	false with RIDE50: ₹208.95 at 15.3 km, ₹157.5 at 15.4 km (the ₹50 discount starts at fare ₹200)	a business rule to confirm; metamorphic properties must exclude promo rides

What happens if the old suite is run unchanged

The 56-test Task 9 hybrid suite was run on calculate_fare2 with its original expected values. **47 of 56 tests fail (84%)**, all because of the 5% GST, which changes every valid fare; these failures are false alarms and tell the team nothing about the new rules. After the expected values were regenerated from the new oracle (the only way to make the suite usable) the suite passes on the correct function but detects only **3 of 12** seeded new-rule faults (GST rate N3, cap after GST N11, wallet check before GST N12), because its inputs never set surge, pooling, tier, toll or airport.

10.3 Evaluation of the original suite

Measure	Task 9 (old rules)	New rules, old suite
V(G) of the function	13	37
Basis paths (V(G) tests)	13 (9 at decision level)	37; the old 9/13 basis tests cover only a quarter of the basis
Tests in suite C (run time at 0.5 s)	56 (28 s)	56 (28 s) but valid only for the old interface

Measure	Task 9 (old rules)	New rules, old suite
New-rule faults detected (of 12)	–	3 (25%)
Full path coverage	97 paths (feasible 41)	≈ 13,824 structural paths: 6,912 s ≈ 1.9 h (> 10 min)

10.4 Three adaptation strategies, applied and measured

Twelve new-rule faults were seeded (N1–N12) in calculate_fare2, covering surge, pooling, loyalty, time-window promo, toll/airport, GST, cap, wallet check and the order of operations. Every strategy was executed on all twelve; a fault counts as found when a test or property fails on the mutant and the same test passes on the correct function (the correct function passed all checks).

ID	Fault
N1	surge limit 3.0 -> excluded (>= 3.0)
N2	surge applied after toll/discounts
N3	GST 5% -> 4%
N4	Gold discount 6% -> 5%
N5	pool of 3: 25% -> 20%
N6	pooled == 2 -> pooled >= 2
N7	airport charge only with toll
N8	time window hour<10 -> hour<=10
N9	time-window threshold >=150 -> >150
N10	loyalty applied before promo
N11	cap applied after GST
N12	wallet check before GST

Strategy A – Retain and prioritise

The 17-test minimised suite C′ is kept and run first, ranked by risk and recent change. Every ride is affected by the changed lines (GST), so the change-based ranking is flat and the 17 tests are ordered by risk (cap, promo, wallet first). Expected values regenerated by oracle2.

Strategy B – Modify with decision tables and orthogonal/pairwise arrays

Kept: the 21 two-value boundary tests (hour, km, wait). Added: 16 decision-table tests for the new rules (promo × tier × pool stacking, the time-window edges 6/7/9/10 and fare 150, surge limits 0.9/1.0/3.0/3.1 and 1.05, pool = 4, negative toll, invalid tier, toll + airport + cap together) and a pairwise suite of 33 tests over 11 parameters. Pairwise reduces the Cartesian product 290,304 to 33 tests (612 pairs, all covered). A final top-up test (9 km, wait 13, hour 8, MORNING20: fare exactly ₹150) was added after the first run showed N9 alive; with it the suite is called B+ (71 tests).

Strategy C – Redesign with model-based and metamorphic testing

Hypothesis generates valid inputs from the parameter domain (100 derandomised examples), the oracle2 model gives the expected value, and eight metamorphic relations are checked without an oracle, on rides without promo (the promo creates a cliff, Section 10.2): R1 surge +0.1 never lowers the fare; R2 a longer ride (+0.1 km) never costs less; R3 a pooled ride is never dearer than a solo ride and pooling of 3 is never dearer than 2; R4 a higher loyalty tier is never dearer; R5 a toll of +₹10 adds exactly ₹10.50 (with GST); R6 the airport charge changes the fare by exactly ₹105; R7 a night ride is never cheaper than a day ride. The correct function passed all properties.

Fault	Old suite (rebaselined)	A Retain (17)	B Modify (B+, 71)	C Redesign (oracle + MR)	of which found by MR alone
N1	MISSED	MISSED	found	found	found
N2	MISSED	MISSED	found	found	found
N3	found	found	found	found	found
N4	MISSED	MISSED	found	found	–

Fault	Old suite (rebaselined)	A Retain (17)	B Modify (B+, 71)	C Redesign (oracle + MR)	of which found by MR alone
N5	MISSED	MISSED	found	found	–
N6	MISSED	MISSED	found	found	–
N7	MISSED	MISSED	found	found	found
N8	MISSED	MISSED	found	MISSED	–
N9	MISSED	MISSED	found	MISSED	–
N10	MISSED	MISSED	found	found	–
N11	found	found	found	found	found
N12	found	found	found	MISSED	–

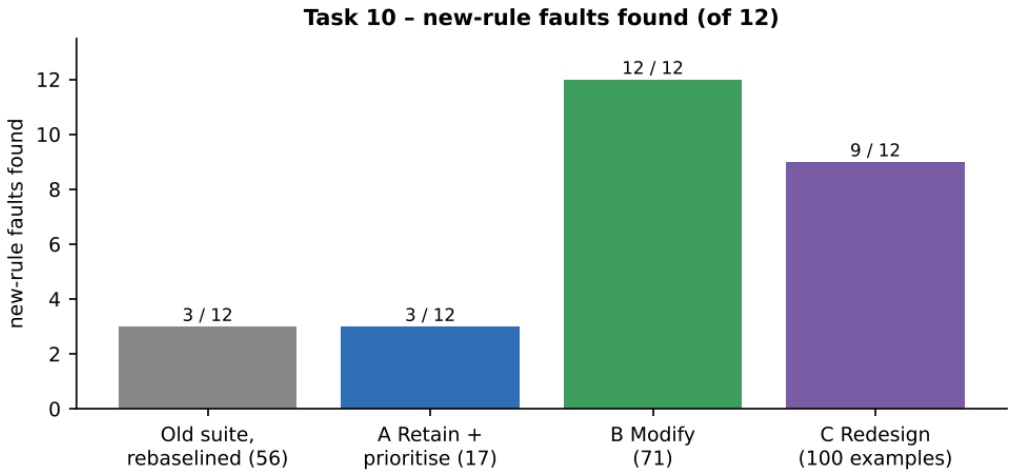


Figure 10.1 – New-rule faults found

Measure	A Retain + prioritise	B Modify (B+)	C Redesign (100 examples)
Faults found (of 12)	3	12	9 (MR alone: 5)
Test executions (service calls)	17	71	992
In-process run time	0.000 s	0.000 s	0.11 s
Projected run time at 0.5 s per test	8 s	35.5 s	496 s
Within 10 minutes (600 s)?	yes (8.5 s)	yes (35.5 s)	yes (496 s) – but 300 examples need 1471 s ≈ 24.5 min: NO
Maintenance effort (estimate)	re-baseline 17 expectations ≈ 1 h per release; misses most new rules	update decision table and oracle ≈ 3.5 h once, pairwise regenerated in minutes; expectations from oracle	model and properties ≈ 6 h once; a rule change is one edit of the model; properties need review when the business rules change (promo cliff)

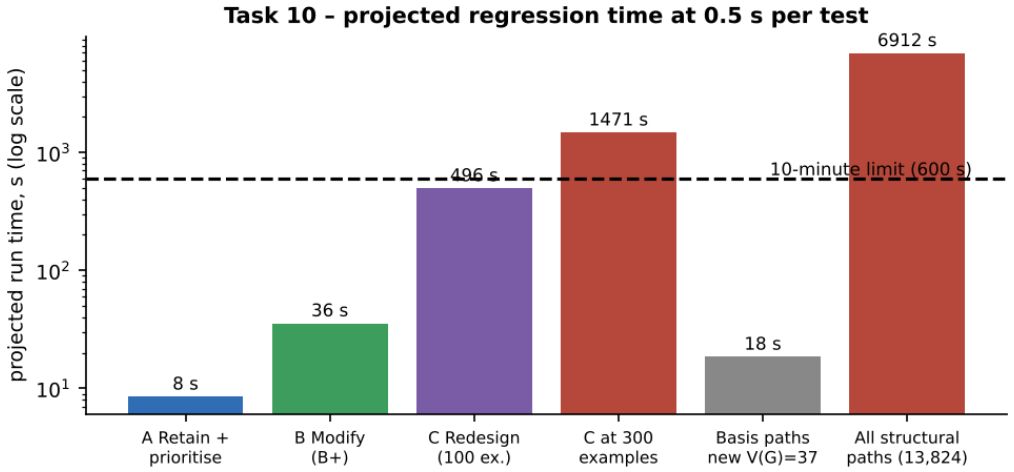


Figure 10.2 – Projected run time against the 10-minute limit (0.5 s per test, assumption)

Misses explained: N8 (hour < 10 → hour <= 10) needs a MORNING20 test at hour 10 with a fare ≥ 150, found by B through the decision-table tests but not by 100 random examples; N9 (fare ≥ 150 → > 150) needs a fare of exactly ₹150, which neither random generation nor pairwise reaches – B found it only after the top-up test; N12 (wallet check before GST) needs a balance between fare and fare × 1.05, found by B (tests with the balance just below the fare) and not by C because random balances 0–1500 rarely fall in that 5% band. Metamorphic properties alone found only 5 of 12: they cannot see a wrong constant that keeps the ordering (GOLD 5% instead of 6%, N4) – the oracle comparison is what finds those.

10.5 Compare and adapt – decision

Criterion	A Retain	B Modify	C Redesign
Faults found	3 / 12	12 / 12	9 / 12
Tests	17	71	100 generated examples (992 calls)
Run time against 10 min	8.5 s	35.5 s	496 s (83% of the budget)
Maintenance	cheap but ineffective	moderate; oracle shared with C	lowest per rule change; highest set-up cost
Decision	not enough	**chosen**	add as a nightly complement

Decision: MODIFY. The Task 9 suite cannot be retained (it detects 3 of 12 new-rule faults and 47 of its 56 expectations are stale). A full REDESIGN is not needed for the gate: the modified suite B+ finds all 12 faults in 35.5 s (6% of the 10-minute budget), keeps the boundary tests that proved their value in Tasks 4 and 9, and reuses the oracle. The revised suite is: (1) the 21 boundary tests from Task 4, (2) 16 decision-table tests for the new rules, (3) 33 pairwise tests over the 11 parameters, (4) one top-up test for the 150 threshold = 71 tests; expected values generated by oracle2, not typed; (5) the metamorphic properties of strategy C run nightly with 300 examples (about 25 minutes, outside the gate), and (6) the 17-test C′ pack kept as the quick smoke test.

10.6 Evidence-based defence

- **Calculation 1 (budget):** 10 min = 600 s; at 0.5 s per test, 600 / 0.5 = 1,200 tests fit. B+ uses 71 (5.9%); C at 100 examples uses 992 calls (83%) and at 300 examples 2,942 calls (245%).
- **Calculation 2 (paths):** the new function has V(G) = 37 and about 13,824 structural paths, so path coverage (6,912 s) is 11.5 times the budget; only basis or pairwise coverage is affordable.
- **Calculation 3 (pairwise reduction):** 290,304 combinations reduced to 33 tests, a factor of 8,797, with all 612 pairs covered.
- **Task 9 measurements:** boundary tests found 6 of 10 faults and pairwise 3 of 10 alone (so both are kept); the minimised pack found 10 of 10 – the design principle “black-box first, white-box only for gaps” still holds, but its input set must now include surge, tier, pool, toll and airport.
- **Condition under which the decision no longer holds:** if the average automated test takes more than 600 / 71 = **8.5 s** (for example, if every test needs a database reset), B+ no longer fits in 10 minutes, and

the gate must fall back to a risk-based subset or to the cheaper model-based generation. The decision also stops holding if a release adds an interaction among three or more new parameters, which pairwise coverage cannot guarantee.

10.7 AI-assisted test generation – benefit and risk

Where an LLM helps	Evidence from this report	Risk if its output is accepted without review
Draft parameter lists and classes for new rules (surge, tier, pool)	LLM gave 16 of the 24 classes in Task 3 (all valid ones, no invalid ones)	missing invalid classes and boundaries; six validation defects would have stayed hidden
Draft decision tables and state diagrams	State diagram correct in Task 5, but the 32-rule table kept 8 impossible rules and no cap	impossible tests that cannot run; missing rules give false confidence
Write pytest/Hypothesis code quickly (17 tests in seconds)	LLM suite had 100% coverage with 4 of 10 faults (Task 9)	coverage alone passes the review while limits are untested
Propose metamorphic relations	the relations in strategy C were checked by running; the “longer never costs less” relation was false for promo rides	a plausible but false property produces false failures, or is “fixed” by weakening it
Fill expected values	values read from the code reproduce the code’s faults (Task 1)	tests pass on faulty code; the oracle is no longer independent

Policy: AI-generated tests are admitted only after (1) expected values come from an independent oracle, (2) the suite is run against the mutant set and the surviving mutants are reviewed, (3) boundary values of every changed rule are present by inspection, (4) traceability to a rule id is added by hand. These checks are cheap (the mutation run takes seconds) and are the reason the report treats an LLM as a draftsman, not as the test designer.

Appendix A – Evidence of Execution (Tool Output)

A.1 Final regression pack under pytest and coverage.py

Command: python -m coverage run --branch --include=listing1.py -m pytest test_fare.py -v, followed by python -m coverage report -m. The 17 tests are the minimised hybrid suite C′ of Task 9; expected values come from the independent oracle and are written as literals in test_fare.py.

```
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0 -- python3
hypothesis profile 'default'
rootdir: swiftride
collecting ... collected 17 items

test_fare.py::test_fare[20-12-2-RIDE50-10-UPI-275.0] PASSED              [  5%]
test_fare.py::test_fare[6-0-23--500-WALLET-122.5] PASSED                 [ 11%]
test_fare.py::test_fare[1.5-3-12-RIDE50-1000-WALLET-50.0] PASSED         [ 17%]
test_fare.py::test_fare[60-30-23--800-WALLET-800.0] PASSED               [ 23%]
test_fare.py::test_fare[0-3-12--1000-CASH-INVALID] PASSED                [ 29%]
test_fare.py::test_fare[61-3-12--1000-CASH-INVALID] PASSED               [ 35%]
test_fare.py::test_fare[6-3-12---5-WALLET-WALLET_LOW] PASSED             [ 41%]
test_fare.py::test_fare[6-0-5--0-CASH-98.0] PASSED                       [ 47%]
test_fare.py::test_fare[14-12-12-RIDE50-500-UPI-150.0] PASSED            [ 52%]
test_fare.py::test_fare[0.1-0-12--0-CASH-50.0] PASSED                    [ 58%]
test_fare.py::test_fare[2.1-0-12--0-CASH-51.2] PASSED                    [ 64%]
test_fare.py::test_fare[10.1-0-12--0-CASH-147.0] PASSED                  [ 70%]
test_fare.py::test_fare[6-6-12--0-CASH-100.0] PASSED                     [ 76%]
test_fare.py::test_fare[6-0-4--0-CASH-122.5] PASSED                      [ 82%]
test_fare.py::test_fare[6-0-22--0-CASH-98.0] PASSED                      [ 88%]
test_fare.py::test_fare[9.5-0-12--0-CASH-140.0] PASSED                   [ 94%]
test_fare.py::test_fare[14.5-9-12-RIDE50-0-CASH-199.0] PASSED            [100%]

============================== 17 passed ==============================
Name          Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------
listing1.py      19      0     16      0   100%
---------------------------------------------------------
TOTAL            19      0     16      0   100%
```

A.2 Mutation run of the regression pack (first 12 mutants and every survivor)

```
mutant  line  change              status
M1      2     <= -> <             killed
M2      2     > -> >=             killed
M3      2     or -> and           killed
M4      2     0 -> 1              killed
M5      2     0 -> -1             killed
M6      2     60 -> 61            killed
M7      2     60 -> 59            killed
M8      4     <= -> <             SURVIVED
M9      4     2 -> 3              killed
M10     4     2 -> 1              killed
M11     5     50 -> 51            killed
M12     5     50 -> 49            killed
M13     6     <= -> <             SURVIVED
M32     10    > -> >=             SURVIVED
M34     10    5 -> 4              SURVIVED
M61     16    > -> >=             SURVIVED
M62     16    800 -> 801          SURVIVED
M63     16    800 -> 799          SURVIVED
...
mutants: 69  killed: 62  survived: 7
```

The seven survivors are the equivalent mutants identified in Task 6; the two real survivors of the combined black-box suite (M15 and M58) are killed by the two top-up tests in this pack, which is why 62 of 69 mutants are killed here.

A.3 Equivalence check of the literal boundary mutants (Section 0.3)

```
km <= 2 -> km < 2      : equivalent over 4,303,728 inputs
km <= 10 -> km < 10    : equivalent over 4,303,728 inputs
wait > 5 -> wait >= 5  : equivalent over 4,303,728 inputs
```

```
km <= 2 -> km <= 2.1   : differs (first difference after 178,466 inputs)
```

A.4 Key code

Differential detection and the independent oracle (common.py, abridged)

```
def killed_by(f, tests):
    """indices of tests whose output differs between the mutant f and the original"""
    return [i for i, t in enumerate(tests) if not same(run(f, t), run(ORIG, t))]

def spec_oracle(km, wait, hour, promo, wallet, mode, strict=False):
    """independent, Decimal-based oracle written only from the specification"""
    km = D(str(km))
    if km <= 0 or km > 60: return "INVALID"
    f = D(50) if km <= 2 else (D(50) + (km - 2) * 12 if km <= 10 else D(146) + (km - 10) * 10)
    f += max(0, wait - 5) * 2
    if hour >= 23 or hour <= 4: f *= D('1.25')
    if promo == "RIDE50" and f >= 200: f -= 50
    f = min(f, D(800))
    if mode == "WALLET" and D(str(wallet)) < f: return "WALLET_LOW"
    return float(f.quantize(D('0.01'), rounding=ROUND_HALF_UP))
```

Ride life cycle with a seeded-fault switch (ride.py, abridged)

```
class Ride:
    def __init__(self, fault=None):
        self.state = "Requested"; self.fee = 0; self.fault = fault
    def accept(self):   self._go(["Requested"], "Accepted")
    def arrive(self):   self._go(["Accepted"], "Arrived")
    def start(self):    self._go(["Arrived"], "Started")
    def complete(self):
        allowed = ["Started"]
        if self.fault == "LF2": allowed = ["Started", "Arrived"]      # seeded fault
        self._go(allowed, "Completed")
    def cancel(self):
        allowed = ["Requested", "Accepted", "Arrived"]
        if self.fault == "LF1": allowed.append("Started")             # seeded fault
        if self.state not in allowed: raise ValueError("illegal cancel")
        self.fee = 30 if self.state == "Arrived" else 0
        self.state = "Cancelled"
```

Metamorphic and model-based property test (Task 10, abridged)

```
@settings(max_examples=100, derandomize=True, database=None, deadline=None)
@given(inputs)                                    # valid inputs of calculate_fare2
def prop(t):
    out = f(*t)
    assert same2(out, oracle2(*t)), "oracle"           # model-based check
    base = f(*no_promo_cash(t))
    assert f(*surge_plus_0_1(t))  >= base - 0.0101     # MR: higher surge never lowers fare
    assert f(*km_plus_0_1(t))     >= base - 0.0101     # MR: longer ride never cheaper (no promo)
    assert abs(f(*toll_plus_10(t)) - base - 10.5) <= 0.0201   # MR: toll + GST is linear
    assert abs(abs(f(*toggle_airport(t)) - base) - 105) <= 0.0201
```

Appendix B – Mapping of the Report to the Assignment Requirements

Task (marks)	Required items	Where answered
1 (4)	12 tests per strategy; faults found, per test, effort; what each misses; input dependence; combination; LLM critique	1.2–1.7
2 (4)	40 tests × 3 approaches; 10 seeds mean/spread; boundary probability; oracle; role of random testing; reliability statement	2.1–2.6
3 (5)	classes with ids; suites A, B, C; 7 faults; assumption breakdown; evaluation and split rule; LLM critique	3.1–3.7
4 (5)	boundary table with precision; counts of three methods; 6 faults; floating point; evaluation; complaint judgement	4.1–4.6
5 (5)	decision table (≥5 conditions); state diagram, table, 6+ invalid transitions; pairwise suite and reduction; seeded faults; evaluation; LLM critique	5.1–5.6
6 (4)	coverage with coverage.py; ≥20 mutants, score, survivors, equivalents; 100%-coverage counter-example; limits; criterion; critique	6.1–6.6
7 (5)	decisions and conditions; minimum tests for lines 14 and 18; three suites with coverage; DC-miss example; short circuit; cost–benefit; LLM MC/DC check	7.1–7.8
8 (4)	three CFGs; V(G) by three methods; granularity; independent paths with data; standard; LLM CFG check	8.1–8.7
9 (6)	three suites with 10 faults and metrics; basis paths and infeasible paths; maintainability; predictions vs measured; workflow diagram and decision table; two other programs; fourth option	9.1–9.9
10 (8)	extended prototype; impact analysis; original-suite evaluation; A/B/C strategies; decision RETAIN/MODIFY/REDESIGN; defence; AI risks	10.1–10.7

Appendix C – Limitations and Honest Notes

- **Fault-injection experiments are small.** Each experiment uses 5–12 seeded faults chosen by the author; the counts (for example 10 of 10) describe these faults on this code, not a general detection rate.
- **Design effort figures** are the author's estimates, not measured times. The in-process run times are measured.
- **LLM results** are one-pass outputs from one model; another run or model may differ. They should be repeated with the tool named in your course and the screenshot attached if the rubric demands it.
- **Run time against 10 minutes** uses an assumed 0.5 s per automated test through the service layer; the function itself runs in microseconds. Change the assumption in Section 10.4 to see the effect (break-even 8.5 s per test for B+).
- **Infeasible paths** were found on a sampled grid and justified analytically; the maximum-fare calculations are exact, the grid is not an exhaustive proof.
- **Equivalent mutants** were classified by exhaustive comparison on the 0.1 km grid with integer wait/hour and a small set of wallet values, which is the input domain of the specification.
- **Assumptions in Task 10** (discount percentages, order of operations, MORNING20 window) are mine; the assignment does not fix them.

Appendix D – Files Delivered with this Report

File	Content
listing1.py, listing2.py	Listing 1 and Listing 2 exactly as printed
common.py, mutants.py, equiv_check.py, covlib.py	differential harness, oracle, mutation generator, equivalence check, coverage measurement
exp1.py … exp10.py	the ten experiments (one script per task) and their JSON results r1.json … r10.json
ride.py	ride life-cycle class with seeded faults (Task 5)
fare2_src.py	extended fare function with the new rules and its 12 seeded faults (Task 10)
test_fare.py	the 17-test regression pack for pytest